

The Sage Programming Language

SageLang

SageLang project

Contents

Part I: Language Fundamentals

Chapter 1: Introduction	1
Chapter 2: Basic Syntax	5
Chapter 3: Control Flow	15
Chapter 4: Functions	25
Chapter 5: Modules and Packages	35
Chapter 6: Error Handling	45
Chapter 7: Concurrency	55
Chapter 8: I/O and Networking	65
Chapter 9: Testing	75
Chapter 10: Performance	85
Chapter 11: Security	95
Chapter 12: Deployment	105
Chapter 13: Documentation	115
Chapter 14: Community	125
Chapter 15: Future	135
Chapter 16: Appendix A: Reference	145
Chapter 17: Appendix B: Glossary	155
Chapter 18: Appendix C: Index	165

Chapter 1: Introduction	1
Chapter 2: Basic Syntax	5
Chapter 3: Control Flow	15
Chapter 4: Functions	25
Chapter 5: Modules and Packages	35
Chapter 6: Error Handling	45
Chapter 7: Concurrency	55
Chapter 8: I/O and Networking	65
Chapter 9: Testing	75
Chapter 10: Performance	85
Chapter 11: Security	95
Chapter 12: Deployment	105
Chapter 13: Documentation	115
Chapter 14: Community	125
Chapter 15: Future	135
Chapter 16: Appendix A: Reference	145
Chapter 17: Appendix B: Glossary	155
Chapter 18: Appendix C: Index	165

Part I: Language Fundamentals

\newpage

Introduction

Sage is a clean, indentation-based systems programming language built in C. It combines the readability of Python with the performance of compiled languages, offering multiple compilation backends, a compile-time safety system inspired by Rust, and a self-hosted compiler written in Sage itself.

Key Features

- **Python-like syntax** with indentation-based blocks and explicit keywords
- **9 execution backends**: C codegen, LLVM IR, native x86-64/aarch64/rv64 assembly, bytecode VM, JIT, AOT, Kotlin/Android
- **Compile-time safety**: ownership tracking, borrow checker, lifetimes, Option types
- **Self-hosted compiler** written in Sage (lexer, parser, interpreter, codegen) with dispatch-table optimizations
- **Rich standard library**: 23+ modules including math, io, string, sys, thread, JSON, collections
- **Object-oriented programming** with classes, inheritance, structs, enums, and traits
- **Generators and async**: yield-based generators, async/await concurrency
- **Three GC modes**: tracing (concurrent mark-sweep), ARC (reference counting), ORC (optimized RC with cycle detection)
- **Kotlin/Android transpiler**: `--emit-kotlin` and `--compile-android` generate complete Gradle projects from a single `.sage` file
- **Developer tooling**: formatter, linter, type checker, LSP server, REPL
- **Bare-metal and OS development**: Multiboot2, UEFI, QEMU integration
- **GPU programming**: Vulkan and OpenGL backends (covered in separate guides)
- **Machine learning**: neural networks, model training (covered in separate guides)
- **Performance library** (`lib/perf.sage`): dispatch tables, signal singletons, flat env cache, shape constructors
- **SageMetal VM**: freestanding bytecode interpreter for bare-metal (no malloc, no libc, no OS)
- **Metal stdlib** (`lib/metal/`): serial, GPIO, IRQ, timer, MMIO for kernel/embedded development
- **Default hybrid runtime**: JIT profiling on hosted, AST on bare-metal, automatic selection
- **v4.0.0 updates**: OIS integration, $O(1)$ dictionary size, $O(N)$ unique checks (simple types), native array reversal, and binary exponentiation for repeating (linear output work).
- **327 interpreter tests**, 1623 self-hosted tests (2060+ total)

Quick Start

```
# Build from source
git clone https://github.com/Night-Traders-Dev/SageLang.git
cd SageLang
make

# Run a program
./sage hello.sage

# Interactive REPL
./sage
```

A minimal Sage program:

```
print "Hello, World!"
```

A slightly larger program demonstrating functions and control flow:

```
proc fizzbuzz(n):
  for i in range(1, n + 1):
    if i % 15 == 0:
      print "FizzBuzz"
    elif i % 3 == 0:
      print "Fizz"
    elif i % 5 == 0:
      print "Buzz"
    else:
      print i
  end
end

fizzbuzz(20)
```

\newpage

Variables and Types

Sage supports `<font face="mono" size="8.4">let</font>` for variable declaration and `<font face="mono" size="8.4">var</font>` for mutable variables.

Variable Declaration

```
# Immutable binding
let name = "Alice"
let age = 30
let pi = 3.14159
let active = true
let nothing = nil

# Mutable variable
var counter = 0
counter = counter + 1
print counter    # 1
```

Variables declared with `<font face="mono" size="8.4">let</font>` cannot be reassigned. Use `<font face="mono" size="8.4">var</font>` when you need a variable whose value changes over time.

Built-in Types

```
| | | T | y | p | e | | | | | | | E | x | a | m | p | l | e | | | | | | | | | | | | | | |
| D | e | s | c | r | i | p | t | i | o | n | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | N | u | m | b | e | r | | | | | | | 4 | 2 | | | | | 3 | . | 1 | 4 | | | | | 0 | x | F
| F | | | | | | 0 | o | 7 | 5 | 5 | | | | | I | n | t | e | g | e | r | | o | r | | f | l | o | a
t | i | n | g | - | p | o | i | n | t | | | | | | | | | | | | | | | | | | | | |
| | | S | t | r | i | n | g | | | | | | | " | h | e | l | l | o | " | | | | | | | | | | | | |
| T | e | x | t | | w | i | t | h | | e | s | c | a | p | e | | s | e | q | u | e | n | c | e | s | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | B | o | o | l | e | a | n | | | | | | | t | r | u | e | | | | | | | f | a | l | s | e | | | |
| L | o | g | i | c | a | l | | v | a | l | l | u | e | s | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | N | i | l | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| A | b | s | e | n | c | e | | o | f | | v | a | l | l | u | e | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | A | r | r | a | y | | | | | | | [ | 1 | , | | 2 | , | | 3 | ] | | | | | | | | | |
| O | r | d | e | r | e | d | , | | m | u | t | a | b | l | e | | c | o | l | l | e | c | t | i | o | n | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | D | i | c | t | | | | | | | { | " | a | " | : | | 1 | } | | | | | | | | | |
| K | e | y | - | v | a | l | l | u | e | | m | a | p | p | l | i | n | g | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | T | u | p | l | e | | | | | | | ( | 1 | , | | " | x | " | ) | | | | | | | | | |
| I | m | m | u | t | a | b | l | e | | o | r | d | e | r | e | d | | p | a | i | r | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | B | y | t | e | s | | | | | | | b | y | t | e | s | ( | 1 | 0 | ) | | | | | | | | | |
| B | i | n | a | r | y | - | s | a | f | e | | b | y | t | e | | b | u | f | f | e | r | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | V | M | | P | r | o | g | r | a | m | | | | | ( | r | e | s | u | l | t | | o | f | | a | s | m | )
| | | | | | C | o | m | p | i | l | e | d | | b | y | t | e | c | o | d | e | | | | | | | | | |
```

Sage numbers are IEEE 754 double-precision floating point. Integer values are exact up to 2^{53} . Hex literals (`<font face="mono" size="8.4">0xFF</font>`) and octal literals (`<font face="mono" size="8.4">0o755</font>`) are supported as of v1.4.

Type Checking

The `<font face="mono" size="8.4">type()</font>` function returns the type of any value as a

string:

```
print type(42)           # "number"
print type("hello")     # "string"
print type(True)        # "bool"
print type(None)        # "nil"
print type([1, 2])      # "array"
print type({"a": 1})     # "dict"
print type((1, 2))      # "tuple"
```

Type Conversion

```
# Number to string
print str(42)           # "42"
print str(3.14)         # "3.14"
print str(True)         # "true"
print str(None)         # "nil"

# String to number
print tonumber("123")   # 123
print tonumber("3.14") # 3.14

# Truncate to integer
print int(3.7)          # 3
print int(-2.9)         # -2

# Character conversions
print chr(65)           # "A"
print ord("A")          # 65
print chr(10)           # newline character
```

\newpage

Operators

Arithmetic Operators

```
print 5 + 2      # 7   (addition)
print 5 - 2      # 3   (subtraction)
print 5 * 2      # 10  (multiplication)
print 5 / 2      # 2.5 (division)
print 5 % 2      # 1   (modulo, uses fmod for floats)
print -5         # -5  (negation)

# Operator precedence follows standard math rules
print 2 + 3 * 4   # 14 (not 20)
print (2 + 3) * 4 # 20
```

The `%` operator uses `fmod()` internally, so it preserves float semantics:

```
print 7.5 % 2.5    # 0
print 3.7 % 1.0    # 0.7
```

Comparison Operators

```
print 1 == 1      # true
print 1 != 2      # true
print 1 < 2       # true
print 2 > 1       # true
print 1 <= 1      # true
print 1 >= 1      # true
```

Comparison works structurally for arrays, dicts, and class instances. Two instances with the same fields and values compare as equal.

Logical Operators

```
print true and false # false
print true or false  # true
print not true       # false
```

Important: In Sage, `0` is **truthy**. Only `false` and `nil` are falsy. Use explicit comparisons: `if x == 0:` rather than `if not x:`.

Bitwise Operators

```
print 5 & 3      # 1  (AND)
print 5 | 3      # 7  (OR)
print 5 ^ 3      # 6  (XOR)
print ~5         # -6 (NOT)
print 1 << 4     # 16 (left shift)
print 16 >> 2    # 4  (right shift)
```

String Concatenation

Strings are concatenated with `<font face="mono" size="8.4">+</font>`:

```
let greeting = "Hello" + ", " + "World!"
print greeting      # Hello, World!

# Convert other types before concatenation
let msg = "Count: " + str(42)
print msg           # Count: 42
```

\newpage

Control Flow

If / Elif / Else

Conditional blocks use `if`, `elif`, `else`, and are terminated with `end`:

```
let x = 42

if x > 100:
  print "big"
elif x > 10:
  print "medium"
else:
  print "small"
end
```

The `elif` keyword supports unlimited branches. Sage processes them as chained if/else structures internally.

```
let grade = 85

if grade >= 90:
  print "A"
elif grade >= 80:
  print "B"
elif grade >= 70:
  print "C"
elif grade >= 60:
  print "D"
else:
  print "F"
end
```

While Loops

```
var i = 0
while i < 5:
  print i
  i = i + 1
end
```

For Loops

```
# Iterate over arrays
let fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print fruit
end

# Range-based for loop
for i in range(5):
    print i    # 0, 1, 2, 3, 4
end

# Range with start and end
for i in range(2, 5):
    print i    # 2, 3, 4
end
```

Break and Continue

```
# Break exits the loop
var i = 0
while true:
    if i == 5:
        break
    end
    print i
    i = i + 1
end

# Continue skips to the next iteration
for x in range(10):
    if x % 2 == 0:
        continue
    end
    print x    # 1, 3, 5, 7, 9
end
```

Match / Case

Pattern matching with optional guards:

```
let color = "red"
match color:
    case "red":
        print "Stop"
    end
    case "yellow":
        print "Caution"
    end
    case "green":
        print "Go"
    end
    default:
        print "Unknown color"
    end
end
```

Match with guards using `<font face="mono" size="8.4">if</font>`:

```
let score = 85
match score:
  case score if score >= 90:
    print "Excellent"
  end
  case score if score >= 70:
    print "Good"
  end
  default:
    print "Needs improvement"
  end
end
```

\newpage

Functions

Functions are declared with `<font face="mono" size="8.4">proc</font>` and terminated with `<font face="mono" size="8.4">end</font>`.

Basic Functions

```
proc greet(name):  
  print "Hello, " + name + "!"  
end  
  
greet("Alice")    # Hello, Alice!
```

Return Values

```
proc add(a, b):  
  return a + b  
end  
  
let result = add(3, 4)  
print result    # 7
```

Default Parameters

```
proc connect(host, port, ssl):  
  print "Connecting to " + host + ":" + str(port)  
end  
  
proc greet_user(name, greeting):  
  print greeting + ", " + name + "!"  
end  
  
greet_user("Alice", "Hello")    # Hello, Alice!  
greet_user("Bob", "Hi")        # Hi, Bob!
```

Recursion

```
proc factorial(n):  
  if n <= 1:  
    return 1  
  end  
  return n * factorial(n - 1)  
end  
  
print factorial(5)    # 120  
  
proc fibonacci(n):  
  if n <= 1:  
    return n  
  end  
  return fibonacci(n - 1) + fibonacci(n - 2)  
end  
  
print fibonacci(10)   # 55
```

Closures

Functions capture their lexical scope, enabling closures:

```
proc make_adder(n):
  proc add(x):
    return x + n
  end
  return add
end

let add5 = make_adder(5)
print add5(10)    # 15
print add5(20)    # 25
```

Stateful Closures

```
proc make_counter():
  var count = 0
  proc increment():
    count = count + 1
    return count
  end
  return increment
end

let counter = make_counter()
print counter()    # 1
print counter()    # 2
print counter()    # 3
```

First-Class Functions

Functions are values and can be passed as arguments:

```
proc apply(f, x):
  return f(x)
end

proc double(n):
  return n * 2
end

print apply(double, 5)    # 10

# Anonymous-style via closures
let square = proc(x):
  return x * x
end

print square(4)    # 16

# Inline anonymous proc expressions
print apply(proc(x): x * x end, 5)    # 25
```

\newpage

Strings

Sage strings support escape sequences and a rich set of built-in operations.

String Basics

```
let greeting = "Hello, World!"
print len(greeting)    # 13

# Character access by index
print greeting[0]      # "H"
print greeting[7]      # "W"
```

Escape Sequences

As of v1.4, Sage supports the following escape sequences in string literals:

E s c a p e	C h a r a c t e r
\ n	N e w l i n e (L F)
\ t	T a b
\ r	C a r r i a g e r e t u r n
\ \	B a c k s l a s h
\ "	D o u b l e q u o t e
\ '	S i n g l e q u o t e
\ 0	N u l l b y t e
\ a	B e l l
\ b	B a c k s p a c e
\ f	F o r m f e e d
\ v	V e r t i c a l t a b
\ x H H	H e x b y t e (e . g . \
x 4 1 \ = " A ")	

```
print "Line one\nLine two"
print "Column1\tColumn2"
print "She said \\\"hi\\\""
print "Hex: \x48\x65\x6C\x6C\x6F"    # "Hello"
```

String Functions

```
# Case conversion
print upper("hello")      # "HELLO"
print lower("HELLO")      # "hello"

# Trimming
print strip("  hello  ")  # "hello"

# Replacement
print replace("hello", "l", "r")  # "herro"

# Splitting and joining
let parts = split("a,b,c", ",")
print parts[0]             # "a"
print join(parts, "-")     # "a-b-c"
```

String Searching

```
print startswith("hello", "hel")  # true
print endswith("hello", "llo")    # true
print contains("hello world", "world") # true
print indexof("hello", "ll")      # 2
```

Character Conversions

```
print chr(65)    # "A"
print chr(10)    # newline
print ord("A")   # 65
print ord("z")   # 122
```

\newpage

Data Structures

Arrays

Arrays are ordered, mutable collections:

```
let nums = [1, 2, 3, 4, 5]
print nums[0]          # 1
print len(nums)        # 5

# Push and pop
push(nums, 6)
print len(nums)        # 6
let last = pop(nums)
print last              # 6

# Both push() and append() work identically
append(nums, 7)
print len(nums)        # 6

# Slicing
let sub = slice(nums, 1, 3)
print sub               # [2, 3]

# Nested arrays
let matrix = [[1, 2], [3, 4], [5, 6]]
print matrix[1][0]     # 3

# Iteration
var total = 0
for n in nums:
    total = total + n
end
print total
```

Dictionaries

Dictionaries are key-value mappings with string keys:

```
let person = {"name": "Alice", "age": 30}
print person["name"]   # "Alice"

# Assignment
person["email"] = "alice@example.com"

# Check keys
print dict_has(person, "name")    # true
print dict_has(person, "phone")   # false

# Get keys and values
let keys = dict_keys(person)
let vals = dict_values(person)

# Delete a key
dict_delete(person, "age")

# Build incrementally (no multiline dict literals)
let config = {}
config["host"] = "localhost"
config["port"] = 8080
config["ssl"] = true
```

Tuples

Tuples are immutable ordered collections:

```
let point = (3, 7)
print point[0]    # 3
print point[1]    # 7
print len(point)  # 2
```

\newpage

Part II: Advanced Language Features

\newpage

Object-Oriented Programming

Classes

Classes are defined with `class` and terminated with `end`. The `init` method serves as the constructor, and `self` refers to the current instance:

```
class Person:
  proc init(self, name, age):
    self.name = name
    self.age = age
  end

  proc greet(self):
    return "Hi, I'm " + self.name
  end
end

let p = Person("Alice", 30)
print p.name      # Alice
print p.greet()   # Hi, I'm Alice
```

Inheritance

Classes can inherit from a parent class. Use `super` to call parent methods:

```
class Animal:
  proc init(self, name):
    self.name = name
  end

  proc speak(self):
    return "..."
  end
end

class Dog(Animal):
  proc init(self, name, breed):
    super.init(name)
    self.breed = breed
  end

  proc speak(self):
    return "Woof!"
  end
end

let d = Dog("Rex", "German Shepherd")
print d.name      # Rex
print d.breed     # German Shepherd
print d.speak()   # Woof!
```

Note that `super` auto-injects `self` as the first argument, but as of v4.1.5, passing it explicitly is also supported for compatibility: `super.init(args)` or `super.init(self, args)`.

Deep Inheritance

```
class A:
  proc init(self, x):
    self.x = x
  end
end

class B(A):
  proc init(self, x, y):
    super.init(x)
    self.y = y
  end
end

class C(B):
  proc init(self, x, y, z):
    super.init(x, y)
    self.z = z
  end
end

let obj = C(1, 2, 3)
print obj.x    # 1
print obj.y    # 2
print obj.z    # 3
```

Arrow Operator

The `>` operator is an alias for `.`, allowing C-style member access:

```
class Vec3:
  proc init(self, x, y, z):
    self->x = x
    self->y = y
    self->z = z
  end

  proc mag_squared(self):
    return self->x * self->x + self->y * self->y + self->z * self->z
  end
end

let v = Vec3(3, 4, 0)
print v->x          # 3
print v->mag_squared() # 25
```

\newpage

Structs, Enums, and Traits

Structs

Structs define data types with typed fields:

```
struct Point:
  x: Int
  y: Int
end

struct Color:
  r: Int
  g: Int
  b: Int
end
```

Struct definitions create constructor-like classes. Fields are accessed with dot notation.

Enums

Enums define a fixed set of named variants:

```
enum Direction:
  North
  South
  East
  West
end

enum Color:
  Red
  Green
  Blue
end
```

Each variant is accessible as `Direction.North`, `Color.Red`, etc.

Traits

Traits define method signatures that classes can implement:

```
trait Printable:
  proc to_string(self) -> String
end

trait Comparable:
  proc compare(self, other) -> Int
end
```

Traits provide a form of interface specification. Classes implementing a trait should provide all the methods declared in the trait.

\newpage

Exception Handling

Exception handling uses `try`, `catch`, `finally`, and `raise`, with blocks terminated by `end`:

Basic Try/Catch

```
try:
  let result = 10 / 0
catch e:
  print "Error: " + e
end
```

Try/Catch/Finally

The `finally` block always executes, whether or not an exception was raised:

```
try:
  print "Opening resource"
  raise "Something went wrong"
catch e:
  print "Caught: " + e
finally:
  print "Cleanup complete"
end
```

Raising Exceptions

```
proc divide(a, b):
  if b == 0:
    raise "Division by zero"
  end
  return a / b
end

try:
  print divide(10, 0)
catch e:
  print e    # Division by zero
end
```

Nested Exception Handling

```
try:
  try:
    raise "inner error"
  catch e:
    print "Inner: " + e
  end
  print "Outer continues"
catch e:
  print "Should not reach here"
end
```

Custom Exception Objects

You can raise any value as an exception, including dicts for structured errors:

```
proc validate_age(age):
  if age < 0:
    let err = {}
    err["type"] = "ValidationError"
    err["message"] = "Age cannot be negative"
    err["value"] = age
    raise err
  end
  return age
end

try:
  validate_age(-5)
catch e:
  print "Error: " + str(e)
end
```

\newpage

Generators and Iterators

Generator Functions

A function containing `yield` becomes a generator. Use `next()` to advance it:

```
proc count_up(start, limit):
  var i = start
  while i < limit:
    yield i
    i = i + 1
  end
end

let gen = count_up(0, 5)
print next(gen)    # 0
print next(gen)    # 1
print next(gen)    # 2
print next(gen)    # 3
print next(gen)    # 4
```

Yielding Values

`yield` suspends the function and returns a value to the caller. The function resumes from the point after `yield` when `next()` is called again:

```
proc fibonacci_gen():
  var a = 0
  var b = 1
  while true:
    yield a
    let temp = a
    a = b
    b = temp + b
  end
end

let fib = fibonacci_gen()
for i in range(10):
  print next(fib)
end
# Output: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

\newpage

Async and Concurrency

Async Procedures

Sage supports `async proc` for concurrent execution. Use `await` to wait for the result:

```
async proc fetch_data(url):
    # Simulated async work
    return "data from " + url
end

let result = await fetch_data("https://example.com")
print result
```

Threads

The `thread` module provides OS-level threading with mutexes:

```
import thread

var shared = 0
let mtx = thread.mutex()

proc worker(id):
    for i in range(1000):
        thread.lock(mtx)
        shared = shared + 1
        thread.unlock(mtx)
    end
end

let t1 = thread.spawn(worker, 1)
let t2 = thread.spawn(worker, 2)
thread.join(t1)
thread.join(t2)

print shared    # 2000
```

Thread Sleep

```
import thread

print "Starting..."
thread.sleep(1.5)    # Sleep for 1.5 seconds
print "Done!"
```

True Atomic Operations

Sage provides C-level atomic operations via `__atomic` compiler builtins. These are truly atomic — safe for concurrent access from multiple threads without locks.

```
let counter = atomic_new(0)      # Create atomic integer, initial value 0

# These are safe to call from multiple threads simultaneously:
atomic_add(counter, 1)           # Atomic increment
atomic_add(counter, 5)           # Atomic add
let val = atomic_load(counter)   # Atomic read (returns 6)

# Compare-and-swap (CAS):
let ok = atomic_cas(counter, 6, 10) # If counter == 6, set to 10
# ok == true, counter is now 10

# Atomic exchange:
let old = atomic_exchange(counter, 0) # Set to 0, return old value (10)
```

Semaphores

POSIX semaphores for controlling access to a finite number of resources:

```
let sem = sem_new(3)           # 3 permits available

sem_wait(sem)                  # Acquire permit (blocks if none available)
sem_wait(sem)                  # Acquire another
# ... do work with the resource ...
sem_post(sem)                  # Release permit

let ok = sem_trywait(sem) # Non-blocking acquire (returns true/false)
```

SMP and Multicore

CPU topology detection and core affinity:

```
# Detect hardware
let cores = cpu_count()          # Logical CPUs (includes hyperthreads)
let physical = cpu_physical_cores() # Physical cores only
let ht = cpu_has_hyperthreading() # true if SMT/HT detected

# Pin thread to specific core
thread_set_affinity(0)           # Pin to core 0
let core = thread_get_core()     # Which core am I on?
```

Multicore work distribution using `lib/os/smp.sage`:

```
import os.smp

smp.print_topology() # Print CPU info

# Run work on all cores in parallel
let results = smp.on_all_cores(proc(core_id):
    return core_id * core_id
)

# Distribute array work across cores
let items = range(10000)
let sums = smp.parallel_for_cores(items, proc(core_id, slice):
    let total = 0
    for x in slice:
        total = total + x
    return total
)
```

\newpage

Pattern Matching

The `match` statement provides pattern matching with guards.

Basic Matching

```
let status = 404

match status:
  case 200:
    print "OK"
  end
  case 301:
    print "Moved"
  end
  case 404:
    print "Not Found"
  end
  case 500:
    print "Server Error"
  end
  default:
    print "Unknown status: " + str(status)
  end
end
```

Guards

Add conditions to cases with `if`:

```
let value = 42

match value:
  case value if value < 0:
    print "Negative"
  end
  case value if value == 0:
    print "Zero"
  end
  case value if value > 0 and value < 100:
    print "Small positive"
  end
  default:
    print "Large"
  end
end
```

Matching Strings

```
proc http_method(method):  
  match method:  
    case "GET":  
      return "Retrieving resource"  
    end  
    case "POST":  
      return "Creating resource"  
    end  
    case "PUT":  
      return "Updating resource"  
    end  
    case "DELETE":  
      return "Deleting resource"  
    end  
    default:  
      return "Unknown method"  
    end  
  end  
end  
  
print http_method("GET")    # Retrieving resource
```

\newpage

Defer Statements

The `defer` statement schedules a statement to execute when the current scope exits, regardless of how it exits (normal return or exception):

```
proc process_file(path):
  let data = "opened"
  defer print "Closing file"

  print "Processing: " + path
  # ... do work ...
  return data
end

process_file("test.txt")
# Output:
#   Processing: test.txt
#   Closing file
```

Deferred statements execute in LIFO (last-in, first-out) order:

```
proc example():
  defer print "First deferred"
  defer print "Second deferred"
  defer print "Third deferred"
  print "Main body"
end

example()
# Output:
#   Main body
#   Third deferred
#   Second deferred
#   First deferred
```

\newpage

Advanced Examples

High-Level: Web Server Mock

This example demonstrates using dicts, classes, and strings to simulate a small web routing system:

```
class Router:
  proc init(self):
    self.routes = {}
  end

  proc add_route(self, path, handler):
    self.routes[path] = handler
  end

  proc handle(self, request):
    let path = request["path"]
    if dict_has(self.routes, path):
      let handler = self.routes[path]
      return handler(request)
    else:
      return {"status": 404, "body": "Not Found"}
    end
  end
end

let app = Router()

app.add_route("/", proc(req):
  return {"status": 200, "body": "Welcome home!"}
end)

app.add_route("/api/hello", proc(req):
  let name = dict_has(req, "name") ? req["name"] : "Guest"
  return {"status": 200, "body": "Hello, " + name}
end)

let response = app.handle({"path": "/api/hello", "name": "Sage"})
print response["body"] # Hello, Sage
```

Low-Level: Custom Binary Packet

Demonstrates `bytes`, manual memory layout, and hashing:

```

import crypto.hash as hash

proc create_packet(payload_str):
  let payload = hash.string_to_bytes(payload_str)
  let size = len(payload)

  # Packet: [Header: 4][Size: 4][Payload: N][Checksum: 4]
  let buf = bytes(4 + 4 + size + 4)

  # Header "SAGE"
  buf[0] = 83; buf[1] = 65; buf[2] = 71; buf[3] = 69

  # Size (little-endian)
  buf[4] = size & 255
  buf[5] = (size >> 8) & 255
  buf[6] = (size >> 16) & 255
  buf[7] = (size >> 24) & 255

  # Payload
  for i in range(size):
    buf[8 + i] = payload[i]
  end

  # Simple XOR checksum
  var cs = 0
  for i in range(8 + size):
    cs = cs ^ buf[i]
  end
  buf[8 + size] = cs

  return buf
end

let p = create_packet("Hello")
print "Packet size: " + str(len(p))
print "Header char: " + chr(p[0]) # S

```

\newpage

```
# Import entire module
import math
print math.abs(-42)      # 42
print math.sqrt(16)      # 4

# Import specific items
from strings import pad_left
print pad_left("42", 5, "0")  # "00042"

# Import with alias
import arrays as arr
let nums = [1, 2, 3]

# Import with item alias
from math import sqrt as square_root
print square_root(25)      # 5
```

`.sage` file is a module. Top-level definitions are

Any `.sage` file is a module. Top-level definitions are exported automatically:

```
# mylib.sage
let VERSION = "1.0"

proc hello(name):
    return "Hello, " + name
end

proc add(a, b):
    return a + b
end
```

```
# main.sage
import mylib
print mylib.VERSION          # 1.0
print mylib.hello("World")  # Hello, World
print mylib.add(2, 3)       # 5
```

Following native modules are built into the interpreter:

The following native modules are built into the interpreter:

[illegible]

The following bundled Sage modules are in the `lib/` directory:

```
| | | M | o | d | u | l | e | | | | | | | | D | e | s | c | r | i | p | t | i | o | n | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| | | | | a | r | r | a | y | s | | | | | F | u | n | c | t | i | o | n | a | l | | a | r | r | a | y
| | | | | o | p | e | r | a | t | i | o | n | s | | | | | | | | | | | | | |
| | | | | s | t | r | i | n | g | s | | | | | S | t | r | i | n | g | | f | o | r | m | a | t | t | i | n
| g | | a | n | d | | t | r | a | n | s | f | o | r | m | a | t | i | o | n | | | | |
| | | | | d | i | c | t | s | | | | | | | | D | i | c | t | i | o | n | a | r | y | | u | t | i | l | i
| t | i | e | s | | | | | | | | | | | | | | | | | |
| | | | | i | t | e | r | | | | | | | | I | t | e | r | a | t | o | r | | c | o | m | b | i | n | a
| t | o | r | s | | a | n | d | | g | e | n | e | r | a | t | o | r | s | | | | |
| | | | | s | t | a | t | s | | | | | | | | S | t | a | t | i | s | t | i | c | a | l | | f | u | n | c
| t | i | o | n | s | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | u | t | i | l | s | | | | | | | | G | e | n | e | r | a | l | - | p | u | r | p | o | s | e |
| u | t | i | l | i | t | i | e | s | | | | | | | | | | | | | | | | | |
| | | | | a | s | s | e | r | t | | | | | | | | T | e | s | t | i | n | g | | a | s | s | e | r | t | i | o
| n | s | | | | | | | | | | | | | | | | | |
| | | | | m | a | t | h | | | | | | | | E | x | t | e | n | d | e | d | | m | a | t | h | | ( | S
| a | l | g | e | - | l | l | e | v | e | l | | s | u | p | p | l | e | m | e | n | t | ) | | | |
| | | | | j | s | o | n | | | | | | | | F | u | l | l | | J | S | O | N | | p | a | r | s | e | r
| | | a | n | d | | s | e | r | i | a | l | i | z | e | r | | | | | | | |
```

OS and System Modules

```
| | | M | o | d | u | l | e | | | | | | | | D | e | s | c | r | i | p | t | i | o | n | | | | |
| | | | | | | | | | | | | | | | | | | | | | | |
| | | | | o | s | . | s | y | n | c | | | | | S | y | n | c | h | r | o | n | i | z | a | t | i | o | n |
| p | r | i | m | i | t | i | v | e | s | | ( | M | u | t | e | x | e | s | , | | S | e | m | a | p | h | o | r |
| e | s | ) | | | |
| | | | | o | s | . | e | r | r | o | r | | | | | P | O | S | I | X | | e | r | r | o | r | | c | o | d | e
| s | | a | n | d | | | s | t | r | e | r | r | o | r | | | u | t | i | l | i | t | y | | | | |
| | | |
| | | | | o | s | . | s | m | p | | | | | | M | u | l | t | i | c | o | r | e | | w | o | r | k | | d
| i | s | t | r | i | b | u | t | i | o | n | | a | n | d | | t | o | p | o | l | l | o | g | y | | | | |
| | | |
| | | | | o | s | . | c | p | i | o | | | | | | I | n | i | t | r | a | m | f | s | | C | P | I | 0 | | a
| r | c | h | i | v | e | | h | a | n | d | l | i | n | g | | | | | | | |
| | | |
| | | | | o | s | . | t | m | p | f | s | | | | | | I | n | - | m | e | m | o | r | y | | t | e | m | p | o | r
| a | r | y | | f | i | l | l | e | s | y | s | t | e | m | | | | | | | |
| | | |
| | | | | o | s | . | a | l | l | o | c | | | | | | B | u | m | p | , | | f | r | e | e | - | l | l | i | s | t | ,
| | a | n | d | | b | i | t | m | a | p | | p | a | l | g | e | | a | l | l | o | c | a | t | o | r | s | | | |
| | | |
| | | | | o | s | . | v | f | s | | | | | | V | i | r | t | u | a | l | | f | i | l | l | e | s | y | s | t
| e | m | | a | b | s | t | r | a | c | t | i | o | n | | l | a | y | e | r | | | | |
| | | |
| | | | | o | s | . | l | i | n | u | x | . | s | y | s | f | s | | | | | L | i | n | u | x | | / | s | y
| s | | f | i | l | l | e | s | y | s | t | e | m | | i | n | t | e | r | f | a | c | e | | | | |
```

os.errno

The `os.errno` module provides POSIX-standard error constants and a `strerror(err)` utility for human-readable error messages.

```
import os.errno as errno

print errno.ENOENT      # 2
print errno.strerror(2) # "No such file or directory"

# Networking error codes (v4.1.5+)
print errno.ECONNRESET  # 104
print errno.EINPROGRESS # 115
```

os.alloc

Provides memory management for kernel heaps via multiple strategy-specific allocators.

Part III: Safety and Type System

\newpage

Type Annotations

Sage supports optional type annotations on function parameters and return types.

Parameter Types

```
proc add(a: Int, b: Int) -> Int:
  return a + b
end

proc greet(name: String) -> String:
  return "Hello, " + name
end

proc is_even(n: Int) -> Bool:
  return n % 2 == 0
end
```

Optional Types

The `T?` syntax marks a type as optional (may be nil):

```
proc find_user(id: Int) -> String?:
  if id == 1:
    return "Alice"
  end
  return nil
end

let user = find_user(99)
if user != nil:
  print user
end
```

Generic Type Parameters

Type annotations support generic parameters for collections and dotted names for types:

```
let fs: vfs.VFS = vfs.VFS(dev)

proc sum_list(nums: Array[Int]) -> Int:
  var total = 0
  for n in nums:
    total = total + n
  end
  return total
end
```

Type annotations are checked by the type checker (`sage check`) and the safety system but are not enforced at runtime by the interpreter.

\newpage

The Safety System

Sage v4.1.5 includes a compile-time safety system inspired by Rust. It provides ownership tracking, borrow checking, lifetime analysis, Option type enforcement, and fearless concurrency checks.

The safety system is a **static analysis pass** that runs after parsing and before code generation. It does not affect the interpreter.

Safety Modes

```
| | M | o | d | e | | | | | | | | F | l | a | g | | | | | | | | | | | | |
| | | | | | | | D | e | s | c | r | i | p | t | i | o | n | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | |
| | O | f | f | | | | | | | | | | ( | d | e | f | a | u | l | t | ) | | | | |
| | | | | | | | N | o | | s | a | f | e | t | y | | c | h | e | c | k | s | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | |
| | A | n | n | o | t | a | t | e | d | | | | | | | | - | - | s | a | f | e | t | y | | | |
| | | | | | | | O | n | l | y | | c | h | e | c | k | | | @ | s | a | f | e | | | f | u |
n | c | t | i | o | n | s | | | | | | | | | | | | | | | | | | | |
| | S | t | r | i | c | t | | | | | | | | | | - | - | s | t | r | i | c | t | - | s | a | f | e
t | y | | | | | | E | n | f | o | r | c | e | | e | v | e | r | y | t | h | i | n | g | | g | l |
o | b | a | l | l | y | | | | | | | | | | | | | | | | | | | |
```

Ownership and Move Semantics

Variables own their data. When a value is assigned to another variable, ownership is transferred (moved), and the original variable becomes invalid:

```
# In strict safety mode:
let data = [1, 2, 3]
let other = data      # ownership moves to 'other'
# print data          # ERROR: use after move
print other           # OK: [1, 2, 3]
```

Borrow Checker

The borrow checker enforces two rules:

- You can have **any number of immutable borrows** at the same time.
- You can have **exactly one mutable borrow** at a time.
- You cannot have both mutable and immutable borrows simultaneously.

```
# Immutable borrows are fine together
let x = 42
let a = x      # immutable borrow
let b = x      # immutable borrow -- OK

# Mutable borrow is exclusive
var y = 10
# Only one mutable reference at a time
```

Lifetimes

Lifetimes ensure references do not outlive the data they point to:

```

proc get_ref():
  let local = "hello"
  return local    # WARNING: reference may outlive local data
end

```

Option Types

In safe contexts, nil values must be handled explicitly using Option semantics:

```

proc find(items, target):
  for item in items:
    if item == target:
      return item    # Some(item)
    end
  end
  return nil         # None
end

let result = find([1, 2, 3], 2)
# In strict mode, must check before use:
if result != nil:
  print result
end

```

The safety library provides `Some()`, `None()`, `unwrap()`, and `map()` for working with Option types explicitly.

Fearless Concurrency: Send and Sync

The safety system tracks `Send` and `Sync` traits:

- **Send**: A type is `Send` if ownership can be transferred between threads.
- **Sync**: A type is `Sync` if it can be safely shared between threads.

In strict mode, passing a non-Send value to `thread.spawn` is an error.

Unsafe Blocks

Operations that bypass safety checks must be wrapped in `unsafe`:

```

unsafe:
  let ptr = mem_alloc(1024)
  mem_write(ptr, 0, "int", 42)
  let val = mem_read(ptr, 0, "int")
  mem_free(ptr)
end

```

Running the Safety Analyzer

```
# Analyze with default (annotated) mode
./sage --safety program.sage

# Analyze with strict mode
./sage --strict-safety program.sage

# Via the sage command
./sage safety program.sage
```

\newpage

Part IV: Standard Library

\newpage

Built-in Modules

Math Module

The `math` module provides trigonometric, logarithmic, and utility functions:

```

import math

# Trigonometry
print math.sin(math.pi / 2)      # 1.0
print math.cos(0)                # 1.0
print math.tan(math.pi / 4)     # ~1.0
print math.asin(1.0)            # ~1.5708
print math.acos(0.0)            # ~1.5708
print math.atan(1.0)            # ~0.7854
print math.atan2(1.0, 1.0)      # ~0.7854

# Powers and roots
print math.sqrt(16)             # 4
print math.pow(2, 10)           # 1024
print math.log(math.e)          # 1
print math.log10(1000)         # 3
print math.exp(1)               # ~2.71828

# Rounding
print math.floor(3.7)           # 3
print math.ceil(3.2)            # 4
print math.round(3.5)           # 4
print math.abs(-42)             # 42
print math.fmod(7, 3)           # 1

# Min/max/clamp
print math.min(3, 7)            # 3
print math.max(3, 7)            # 7
print math.clamp(150, 0, 100)   # 100

# Checks
print math.isnan(0.0 / 0.0)     # true
print math.isinf(1.0 / 0.0)     # true

# Constants
print math.pi                   # 3.14159265358979
print math.e                    # 2.71828182845905
print math.tau                  # 6.28318530717959
print math.inf                  # Infinity
print math.nan                  # NaN

# Utility
print math.pow_int(2, 10)       # 1024 (binary exponentiation, O(log n))

# Arithmetic Visualization
# Evaluates an expression string and shows step-by-step work.
# Backends: "sage" (default), "c", "asm"
# Controlled by CLI flag --math-work=grade,exec
math.printm("123 + 456", backend="sage", formats=["grade"])
# Output:
# 123
# + 456
# ----
# 579

# Constants
print math.pi                   # 3.14159265358979
print math.e                    # 2.71828182845905
print math.tau                  # 6.28318530717959
print math.inf                  # Infinity
print math.nan                  # NaN

# Random
print math.random()             # random float in [0, 1)

```

IO Module

The `io` module provides file system operations:

```

import io

# Read and write files
let content = io.readfile("data.txt")
io.writefile("output.txt", "Hello, World!")
io.appendfile("log.txt", "New entry\n")

# File system queries
print io.exists("data.txt")      # true/false
print io.isdir("/home")         # true
io.mkdir("new_dir")             # Creates directory (0755 secure permissions)
io.mkpath("a/b/c")              # Creates recursive path
print io.filesize("data.txt")    # file size in bytes

# Binary I/O
let bytes = io.readbytes("image.png")    # Read as Bytes value
print bytes_len(bytes)                   # number of bytes

let buf = bytes("Hello\n")
io.writebytes("output.bin", buf)         # Write Bytes (also accepts Array)
io.appendbytes("log.bin", buf)           # Append Bytes

# Directory listing
let files = io.listdir(".")
for f in files:
  print f
end

# Remove files
io.remove("temp.txt")

```

String Module

The `string` module provides advanced string operations:

```

import string

# Search
print string.find("hello world", "world")    # 6
print string.rfind("hello hello", "hello")   # 6
print string.startswith("hello", "hel")      # true
print string.endswith("hello", "llo")        # true
print string.contains("hello world", "world") # true

# Character access
print string.char_at("hello", 1)              # "e"
print string.ord("A")                         # 65
print string.chr(65)                         # "A"

# Manipulation
print string.repeat("ab", 3)                  # "ababab"
print string.count("banana", "an")           # 2
print string.substr("hello", 1, 3)            # "ell"
print string.reverse("hello")                 # "olleh"

```

Sys Module

The `sys` module provides system-level utilities:

```

import sys

# Command-line arguments
let args = sys.args()
for arg in args:
  print arg
end

# Environment
print sys.platform      # "linux", "darwin", or "windows"
print sys.version       # "3"
print sys.getenv("HOME") # /home/user

# Timing
let start = sys.clock()
# ... do work ...
let elapsed = sys.clock() - start
print "Took " + str(elapsed) + " seconds"

# Sleep
sys.sleep(0.5)    # sleep for 0.5 seconds

# Exit
sys.exit(0)

# Dynamic Call
# Calls a native function or class constructor with arguments.
let res = sys.call(math.sqrt, 16) # 4

# Shell Execution
# exec() returns exit code, shell_exec() returns output string.
let code = sys.exec("ls")
let output = sys.shell_exec("whoami")

```

Thread Module

The `thread` module provides OS-level concurrency:

```

import thread

# Spawn a thread
proc worker(id):
  print "Worker " + str(id) + " running"
end

let t = thread.spawn(worker, 1)
thread.join(t)

# Mutexes for synchronization
let mtx = thread.mutex()
thread.lock(mtx)
# ... critical section ...
thread.unlock(mtx)

# Sleep
thread.sleep(0.1)    # sleep for 100ms

```

\newpage

Collection Libraries

Arrays Module

```
import arrays

let nums = [5, 3, 8, 1, 9, 2]

# Functional operations
let doubled = arrays.map(nums, proc(x): return x * 2 end)
let evens = arrays.filter(nums, proc(x): return x % 2 == 0 end)
let total = arrays.reduce(nums, 0, proc(acc, x): return acc + x end)

# Search
print arrays.contains(nums, 8)      # true
print arrays.index_of(nums, 8)     # 2

# Transform
let rev = arrays.reverse(nums)
let combined = arrays.concat([1, 2], [3, 4])
```

Strings Module

```
import strings

let text = " Hello, World! "
print strings.compact(text)          # "Hello, World!"
print strings.pad_left("42", 5, "0") # "00042"
print strings.pad_right("hi", 10, ".") # "hi....."
print strings.repeat("ab", 3)        # "ababab"
print strings.surround("hello", "[", "]") # "[hello]"
print strings.snake_case("HelloWorld") # "hello_world"
print strings.dash_case("helloWorld")  # "hello-world"
print strings.csv(["a", "b", "c"])     # "a,b,c"
```

Dicts Module

```
import dicts

let config = {"host": "localhost", "port": 8080, "debug": false}

print dicts.size(config)          # 3
print dicts.get_or(config, "timeout", 30) # 30 (default)
print dicts.has_all(config, ["host", "port"]) # true

let entries = dicts.entries(config)
for e in entries:
  print str(e[0]) + " = " + str(e[1])
end
```

Iterator Module

```
import iter

# Infinite counter
for n in iter.count(0, 2):
    if n > 10:
        break
    end
    print n    # 0, 2, 4, 6, 8, 10
end

# Enumeration
let fruits = ["apple", "banana", "cherry"]
for pair in iter.enumerate_array(fruits):
    print str(pair[0]) + ": " + pair[1]
end

# Take first N
let first5 = iter.take(iter.count(0, 1), 5)
```

Stats Module

```
import stats

let scores = [85, 92, 78, 95, 88, 91, 76]

print stats.mean(scores)      # ~86.4
print stats.min_value(scores) # 76
print stats.max_value(scores) # 95
print stats.range_span(scores) # 19
print stats.sum(scores)       # 605
```

Assert Module

```
import assert

let result = 2 + 2
assert.assert_equal(result, 4, "Basic addition")
assert.assert_true(result > 0, "Positive result")
assert.assert_not_nil(result, "Not nil")
assert.assert_close(3.14159, 3.14, 0.01, "Pi approximation")
```

\newpage

JSON Module

The `json` module is a full JSON parser and serializer written in pure Sage, ported from cJSON:

```
import json

# Parse a JSON string
let text = "{\"name\": \"Alice\", \"age\": 30}"
let data = json.cJSON_Parse(text)

# Access values
let name_item = json.cJSON_GetObjectItem(data, "name")
print json.cJSON_GetStringValue(name_item)    # Alice

let age_item = json.cJSON_GetObjectItem(data, "age")
print json.cJSON_GetNumberValue(age_item)     # 30

# Create JSON
let obj = json.cJSON_CreateObject()
json.cJSON_AddStringToObject(obj, "greeting", "hello")
json.cJSON_AddNumberToObject(obj, "count", 42)
json.cJSON_AddBoolToObject(obj, "active", true)

# Serialize to string
let output = json.cJSON_Print(obj)
print output

# Clean up
json.cJSON_Delete(data)
json.cJSON_Delete(obj)
```

The JSON module passes 88 tests and supports all JSON types: objects, arrays, strings, numbers, booleans, and null.

\newpage

Safety Library

The safety library provides explicit Option type operations for working with potentially-nil values:

```
# Option type constructors
let some_val = Some(42)
let no_val = None()

# Unwrap (panics if None)
let x = unwrap(some_val)    # 42

# Safe access with map
let doubled = map(some_val, proc(v): return v * 2 end)
# doubled is Some(84)

# Check before use
if some_val != nil:
  print unwrap(some_val)
end
```

When `--strict-safety` is enabled, the compiler enforces that all Option-typed values are checked before use, preventing nil-related runtime errors.

\newpage

Part V: Compilation and Backends

\newpage

Compilation Backends

Sage provides three compilation backends, each targeting different use cases.

C Backend

The C backend translates Sage to portable C code:

```
# Emit C source code
./sage --emit-c program.sage -o output.c

# Compile to binary via C (uses gcc/cc)
./sage --compile program.sage -o program

# Run the compiled binary
./program
```

The C backend is the most mature and supports the full language. The generated C code is portable and can be compiled with any C11-compliant compiler.

LLVM Backend

The LLVM backend generates LLVM IR text, then uses `clang` to compile:

```
# Emit LLVM IR
./sage --emit-llvm program.sage -o output.ll

# Compile to binary via LLVM
./sage --compile-llvm program.sage -o program
```

The LLVM backend supports classes, methods, exception handling, bitwise operators, and GPU operations. It links against `llvm_runtime.c` which provides 40+ runtime functions.

Native Assembly Backend

The native backend generates assembly directly for three architectures:

```
# Emit x86-64 assembly
./sage --emit-asm program.sage -o output.s --target x86-64

# Emit aarch64 assembly
./sage --emit-asm program.sage -o output.s --target aarch64

# Emit RISC-V 64-bit assembly
./sage --emit-asm program.sage -o output.s --target rv64

# Compile to native binary
./sage --compile-native program.sage -o program --target x86-64
```

Optimization Levels

All backends support optimization passes:

```
./sage -O0 program.sage      # No optimization
./sage -O1 program.sage      # Basic: constant folding
./sage -O2 program.sage      # Standard: + dead code elimination
./sage -O3 program.sage      # Aggressive: + function inlining
```

The optimization passes operate on the AST before code generation:

		L	e	v	e	l				P	a	s	s	e	s														
		`	-	0	0	`				N	o	n	e																
		`	-	0	1	`				C	o	n	s	t	a	n	t		f	o	l	d	i	n	g				
		`	-	0	2	`				C	o	n	s	t	a	n	t		f	o	l	d	i	n	g	,		d	
a	d			c	o	d	e		e	l	i	m	i	n	a	t	i	o	n										
		`	-	0	3	`				C	o	n	s	t	a	n	t		f	o	l	d	i	n	g	,		d	
a	d			c	o	d	e		e	l	i	m	i	n	a	t	i	o	n			i	n	l	i	n	i	n	g

Debug Info

```
./sage -g program.sage      # Include debug information
```

\newpage

Bare-Metal and OS Development

Sage includes a complete pipeline for building bare-metal kernels that boot and run on QEMU for x86_64, aarch64, and RISC-V 64. The boot libraries generate assembly, linker scripts, and QEMU commands from pure Sage code.

Supported Architectures

```
| | | A | r | c | h | i | t | e | c | t | u | r | e | | | | B | o | o | t | | M | e | t | h | o | d | | | | | | | | | | | | | | | | | | |
| | | S | e | r | i | a | l | | U | A | R | T | | | | Q | E | M | U | | M | a | c | h | i | n | e | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | x | 8 | 6 | _ | 6 | 4 | | | | M | u | l | t | i | b | o | o | t | 1 | | ( | E | L | F | 3 | 2 | ) | |
| | | C | 0 | M | 1 | | p | o | r | t | | I | / | 0 | | ( | 0 | x | 3 | F | 8 | ) | | | | d | e | f |
a | u | t | | P | C | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | a | a | r | c | h | 6 | 4 | | | | D | i | r | e | c | t | | E | L | F | | l | o | a | d | | | |
| P | L | 0 | 1 | 1 | | M | M | I | 0 | | ( | 0 | x | 0 | 9 | 0 | 0 | 0 | 0 | 0 | 0 | ) | | | | v | i | r |
t | | + | | c | o | r | t | e | x | - | a | 5 | 7 | | | | | | | | | | | | | | | | | | | | | | | |
| | | r | i | s | c | v | 6 | 4 | | | | D | i | r | e | c | t | | E | L | F | | l | o | a | d | | | |
| N | S | 1 | 6 | 5 | 5 | 0 | | M | M | I | 0 | | ( | 0 | x | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | ) | | | | v |
i | r | t | | ( | - | b | i | o | s | | n | o | n | e | ) | | |
```

Advanced Boot Infrastructure

SageLang v4.1.5 expands the `os.boot` library with 20+ new modules for building sophisticated multi-stage bootloaders.

Firmware Interaction

The `os.boot.bios` module provides code generation for legacy BIOS interrupts from real mode. This allows Sage-based bootloaders to set video modes, read from disks via LBA extensions, and query system information before switching to protected mode.

```
import os.boot.bios as bios

# Set 80x25 text mode
let asm = bios.set_video_mode(0x03)

# Read disk sectors
let dap = bios.disk_address_packet(lba=2048, count=16, dest=0x20000)
asm = asm + bios.int13_read(drive=0x80, "dap_label")
```

Memory Detection

`os.boot.e820` handles the collection and normalization of the system memory map from BIOS. It can find usable RAM regions and determine the highest physical address available for kernel placement.

```
import os.boot.e820 as e820

# Collect memory map entries
let asm = e820.collect_asm("mmap_buffer")

# Filter for usable RAM regions
let usable = e820.filter_usable(mmap)
```

Mode Transitions

The transition from 16-bit real mode to 64-bit long mode is handled by a sequence of composable modules:

- `<b><font face="mono" size="8.4">os.boot.real_mode</font></b>`: Real-mode stack setup and far jumps.
- `<b><font face="mono" size="8.4">os.boot.prot_mode</font></b>`: Entering 32-bit protected mode with a flat GDT.
- `<b><font face="mono" size="8.4">os.boot.long_mode</font></b>`: Transitioning to 64-bit mode via PAE and PML4 setup.

Kernel Loading and Handoff

- `<b><font face="mono" size="8.4">os.boot.elf_load</font></b>`: Parses ELF binaries and copies `<font face="mono" size="8.4">PT_LOAD</font>` segments to their physical load addresses.
- `<b><font face="mono" size="8.4">os.boot.handoff</font></b>`: Defines the SageOS Boot Protocol, allowing the bootloader to pass a structured information block (framebuffer, memory map, ACPI tables) to the kernel entry point.

Architecture-Specific Support

- `<b><font face="mono" size="8.4">os.boot.sbi</font></b>`: RISC-V Supervisor Binary Interface (SBI) wrappers for S-mode bootloaders.
- `<b><font face="mono" size="8.4">os.boot.psci</font></b>`: ARM Power State Coordination Interface for secondary core bring-up.

Quick Start: Build a Kernel with Sage

The `<font face="mono" size="8.4">os.boot.build</font>` module generates all files needed for a bootable kernel. Here is a complete example that builds and runs on all three architectures:

```
# build_kernel.sage – Generate a bootable kernel for any architecture
gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Pick your architecture: "x86_64", "aarch64", or "riscv64"
let arch = "x86_64"

# Generate boot assembly with serial output
let boot_asm = start.generate_boot_asm_mbl("Hello from SageOS!")

# Generate linker script
let linker = build.generate_linker_x86_mbl()

# Write files
io.writefile("boot.S", boot_asm)
io.writefile("linker.ld", linker)

# Print build commands
print "Build:"
print "  as --32 -o boot.o boot.S"
print "  ld -m elf_i386 -T linker.ld -o kernel.elf boot.o"
print "Run:"
print "  " + build.qemu_command("x86_64", "kernel.elf")
```

Example: x86_64 Kernel

Generate boot assembly, compile, and run:

```
gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Generate self-contained 32-bit multiboot1 kernel with serial
let asm = start.generate_boot_asm_mbl("Hello from SageOS on x86_64!")
let ld = build.generate_linker_x86_mbl()

io.writefile("boot.S", asm)
io.writefile("linker.ld", ld)
```

Build and run:

```
# Assemble (32-bit ELF for multiboot1 compatibility)
as --32 -o boot.o boot.S

# Link
ld -m elf_i386 -T linker.ld -o kernel.elf boot.o

# Run in QEMU
qemu-system-x86_64 -m 128M -display none -serial mon:stdio -kernel kernel.elf
# Output: Hello from SageOS on x86_64!
```

The generated boot stub initializes COM1 at 115200 baud (8N1), prints the message over serial, and halts with `<font face="mono" size="8.4">cli; hlt</font>`.

Example: aarch64 Kernel

```
gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Generate aarch64 boot assembly + serial (PL011 at 0x09000000)
let asm = start.emit_start_aarch64("kmain", "stack_top")
asm = asm + build.generate_serial_boot_aarch64()

# Generate C kernel
let kernel_c = build.generate_kernel_c("aarch64", "Hello from SageOS on aarch64!")

io.writefile("boot.S", asm)
io.writefile("kernel.c", kernel_c)
```

Build and run:

```
# Assemble
aarch64-linux-gnu-as -o boot.o boot.S

# Compile kernel C
aarch64-linux-gnu-gcc -ffreestanding -nostdlib -c -o kernel.o kernel.c

# Link (base address 0x40000000 for QEMU virt)
aarch64-linux-gnu-ld -T linker.ld -o kernel.elf boot.o kernel.o

# Run in QEMU
qemu-system-aarch64 -machine virt -cpu cortex-a57 -m 128M \
  -display none -serial mon:stdio -kernel kernel.elf
# Output: Hello from SageOS on aarch64!
```

The boot stub disables interrupts (DAIF), sets up the stack, zeroes BSS, then jumps to `<font face="mono" size="8.4">kmain</font>`. The serial driver uses the PL011 UART with FIFO enabled.

Example: RISC-V 64 Kernel

```
gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Generate riscv64 boot assembly + serial (NS16550 at 0x10000000)
let asm = start.emit_start_riscv64("kmain", "stack_top")
asm = asm + build.generate_serial_boot_riscv64()

# Generate C kernel
let kernel_c = build.generate_kernel_c("riscv64", "Hello from SageOS on riscv64!")

io.writefile("boot.S", asm)
io.writefile("kernel.c", kernel_c)
```

and run:

```
# Assemble
riscv64-linux-gnu-as -march=rv64gc -mabi=lp64d -o boot.o boot.S

# Compile kernel C
riscv64-linux-gnu-gcc -ffreestanding -nostdlib -march=rv64gc \
    -mabi=lp64d -c -o kernel.o kernel.c

# Link (base address 0x80000000 for QEMU virt)
riscv64-linux-gnu-ld -T linker.ld -o kernel.elf boot.o kernel.o

# Run in QEMU (must use -bios none to skip OpenSBI)
qemu-system-riscv64 -machine virt -m 128M \
    -display none -serial mon:stdio -bios none -kernel kernel.elf
# Output: Hello from SageOS on riscv64!
```

boot stub disables machine-mode interrupts (mstatus.MIE), sets up the stack, zeroes BSS, calls `kmain`. The serial driver uses a 0-compatible UART in MMIO mode.

Library Reference

[illegible]

Panel Libraries

[illegible]

```
# Start QEMU paused, waiting for GDB on port 1234
qemu-system-x86_64 -m 128M -display none -serial mon:stdio \
    -kernel kernel.elf -s -S &

# Connect GDB
gdb kernel.elf -ex 'target remote :1234' -ex 'break _start' -ex 'continue'
```

```
# aarch64
aarch64-linux-gnu-gdb kernel.elf -ex 'target remote :1234'

# riscv64
riscv64-linux-gnu-gdb kernel.elf -ex 'target remote :1234'
```

Part VI: Tooling

\newpage

Developer Tools

REPL

Launch the interactive read-eval-print loop:

```
./sage
# or
./sage --repl
```

```
sage> 2 + 3
5
sage> let x = "hello"
sage> print upper(x)
HELLO
sage> proc square(n): return n * n end
sage> print square(7)
49
sage> :quit
```

The REPL supports multiline input, error recovery, and all language features. Commands like `:stats` can be used to view interpreter and GC statistics.

Formatter

Automatically format Sage source files:

```
./sage --fmt program.sage
# or
./sage fmt program.sage
```

The formatter normalizes indentation, spacing, and code layout. Both C and self-hosted (Sage) implementations are available.

Lint

Check code for style issues and common mistakes:

```
./sage --lint program.sage
# or
./sage lint program.sage
```

The linter reports unused variables, naming convention violations, overly complex functions, and other code quality issues.

Type Checker

Run static type analysis:

```
./sage check program.sage
```

The type checker performs best-effort type inference and reports type mismatches, unreachable code, and annotation violations.

Safety Analyzer

Run the compile-time safety analysis:

```
./sage safety program.sage  
./sage --strict-safety program.sage
```

Reports ownership violations, borrow conflicts, lifetime errors, and nil-safety issues.

Language Server (LSP)

Start the LSP server for editor integration:

```
./sage --lsp  
# or  
./sage-lsp
```

The LSP server provides:

- **Autocompletion** of functions, variables, and module members
- **Diagnostics** (errors and warnings as you type)
- **Hover info** (type information and documentation)
- **Go-to-definition** navigation

Both C and self-hosted implementations exist. Compatible with VS Code, Neovim, and any editor supporting the Language Server Protocol.

\newpage

Build System

Makefile Targets

```
make                # Build the sage interpreter
make test           # Run interpreter tests (269+ tests)
make test-selfhost  # Run self-hosted tests (1567+ tests)
make test-all      # Run all tests

# Bare-metal / QEMU
make qemu-bare      # Run kernel in QEMU (x86_64)
make qemu-bare-arm64 # Run kernel in QEMU (aarch64)
make qemu-debug     # Debug kernel with GDB
make kernel-bare    # Compile bare-metal kernel
make kernel-uefi    # Compile UEFI application
```

Version

The version is stored in a single `VERSION` file at the repository root. All build systems (Makefile, CMakeLists.txt, sagemake) read from this file automatically. Current version: **3**.

\newpage

Part VIb: Metaprogramming

\newpage

Compile-Time Execution

Sage supports compile-time code execution through the `<font face="mono" size="8.4">comptime</font>` keyword. Because Sage has a built-in interpreter, any Sage code can run during compilation to produce constants, lookup tables, or configuration data that is baked directly into the binary.

Comptime Blocks

A `<font face="mono" size="8.4">comptime:</font>` block executes its body at compile time. Variables defined inside a comptime block are available to subsequent code.

```
# Pre-compute a lookup table
comptime:
  let SINE_TABLE = []
  for i in range(256):
    push(SINE_TABLE, math.sin(i * math.pi / 128.0))
  end
end

proc get_sine(angle):
  return SINE_TABLE[angle % 256]
end
```

Comptime Expressions

The `<font face="mono" size="8.4">comptime(expr)</font>` form evaluates a single expression at compile time:

```
proc factorial(n):
  if n <= 1:
    return 1
  return n * factorial(n - 1)

# Computed at compile time, baked as a constant
let FACT_10 = comptime(factorial(10))
```

\newpage

Pragmas and Decorators

Pragmas attach metadata to declarations using the `@` symbol. They inform the compiler backends how to handle specific functions, structs, or blocks.

Syntax

```
@pragma_name
@pragma_name("argument")
```

Built-in Pragmas

```
| | | P | r | a | g | m | a | | | | T | a | r | g | e | t | | | | E | f | f | e | c | t | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | @ | i | n | l | i | n | e | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | C | b | a | c | k | e | n | d | | t | o | | e | l | m | i | t | | | | | s | t | a | t | i | c | |
| | | | | i | n | l | i | n | e | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | @ | p | l | a | c | k | e | d | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | # | p | r | a | g | m | a | | p | l | a | c | k | ( | p | l | u | s | h | , | | 1 | ) | | | | f | l | o | r | |
| | | | | n | o | | p | l | a | d | d | i | n | g | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | @ | s | e | c | t | i | o | n | ( | " | n | a | m | e | " | ) | | | | | | | | | | | | | | | |
| | | | | | | | | | P | l | a | c | e | s | | f | u | n | c | t | i | o | n | | i | n | | a | | s | p | e | c | i | f |
| | | | | i | c | | | | | E | L | F | | s | e | c | t | i | o | n | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | @ | a | l | l | i | g | n | ( | " | N | " | ) | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | l | i | g | n | m | e | n | t | | h | i | n | t | | f | l | o | r | | s | t | r | u | c | t | | l | a | y | o |
| | | | | u | t | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | @ | d | e | p | r | e | c | a | t | e | d | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | k | s | | a | | f | u | n | c | t | i | o | n | | a | s | | d | e | p | r | e | c | a | t | e | d | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | @ | n | o | r | e | t | u | r | n | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | a | | f | u | n | c | t | i | o | n | | t | h | a | t | | n | e | v | e | r | | r | e | t | u | r | n |
| | | | | s | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
```

Examples

```
@packed
struct Ipv4Header:
    version_ihl: u8
    tos: u8
    total_length: u16

@inline
proc fast_math(x, y):
    return x * y

@section(".multiboot_header")
proc boot_header():
    pass
```

Multiple Pragmas

Multiple pragmas can be stacked on a single declaration:

```
@inline
@deprecated
proc old_multiply(a, b):
    return a * b
```

\newpage

AST Macros

Macros are functions that execute at compile time and operate on code structure. In the current implementation, macros are defined with the `<font face="mono" size="8.4">macro</font>` keyword and behave as compile-time procedures.

Macro Definition

```
macro log_call(label):  
  print "entering " + label  
  # ... macro body ...  
  print "exiting " + label  
  
log_call("main")
```

In interpreter mode, macros are treated as regular functions. In compiled mode, macro bodies are expanded at the call site during compilation.

Future: Quote and Unquote

The `<font face="mono" size="8.4">quote</font>` and `<font face="mono" size="8.4">unquote</font>` keywords are reserved for future AST macro support, enabling macros that manipulate Abstract Syntax Tree nodes directly:

```
macro safe_zone(body):  
  return quote:  
    sys.enable_strict_safety()  
    defer sys.restore_safety()  
    unquote(body)  
  end
```

\newpage

Generics

Sage supports generic type parameters on procedures and structs using bracket syntax `[T, U, ...]`. In the current implementation, Sage uses dynamic typing, so generic parameters serve as documentation and enable future monomorphization in compiled backends.

Generic Procedures

```
proc identity[T](x: T) -> T:
  return x

proc swap[T](a: T, b: T):
  let tmp = a
  a = b
  b = tmp
```

Generic Structs

```
struct Pair[A, B]:
  first: A
  second: B

struct Stack[T]:
  items: Array[T]
  size: Int
```

Monomorphization (Planned)

When compiled with the C or LLVM backend, the compiler will generate specialized versions for each concrete type used:

```
# At compile time, generates Stack_Int and Stack_String
let int_stack = Stack[Int]()
let str_stack = Stack[String]()
```

\newpage

Part VIc: Garbage Collection

GC Modes

Sage provides three garbage collection strategies, selectable at startup:

```
sage --gc:tracing file.sage  # Default: concurrent mark-sweep
sage --gc:arc file.sage     # Automatic Reference Counting
sage --gc:orc file.sage     # Optimized RC with cycle detection
```

Tracing GC (Default)

Concurrent tri-color mark-sweep with SATB write barriers:

- Sub-millisecond stop-the-world pauses (root scan ~50-200us)
- Adaptive threshold triggering based on allocation pressure
- New objects born BLACK (allocated-black invariant)
- Sweeping in 256-object batches to bound pause times

ARC (Automatic Reference Counting)

Deterministic memory reclamation via reference counts:

- Objects freed immediately when reference count reaches zero
- Predictable memory usage (no deferred collection)
- Convenience macros: `ARC_RETAIN`, `ARC_RELEASE`, `ARC_ASSIGN`
- Cannot collect reference cycles — use ORC for cyclic structures

ORC (Optimized Reference Counting)

Nim-inspired ORC combining ARC with Lins' trial deletion cycle collector:

- Three-phase cycle detection: mark PURPLE candidates, trial-decrement scan, collect WHITE garbage
- Triggers cycle collection every 500 decrements (more aggressive than ARC's 1000)
- Recommended for programs with complex object graphs (linked lists, trees, circular references)

GC API

```
gc_collect()      # Force a collection cycle
gc_stats()       # Print GC statistics
gc_enable()      # Enable automatic collection
gc_disable()     # Disable automatic collection (manual only)
gc_set_arc()     # Switch to ARC mode at runtime
gc_set_orc()     # Switch to ORC mode at runtime
gc_mode()        # Returns "tracing", "arc", or "orc"
```

\newpage

Emitting Kotlin

[illegible]

Building Android Apps

```
Build with: <font face="mono" size="8.4">cd my_app && ./gradlew
assembleDebug</font>
```

Type Specialization (-O2+)

At optimization level 2+, the transpiler emits native Kotlin types for variables initialized with literals:

```
let x = 10          # emits: var x: Double = 10.0
let name = "Sage"  # emits: var name: String = "Sage"
let flag = true    # emits: var flag: Boolean = true
```

This eliminates SageVal boxing overhead on hot paths.

Generators

Generator functions transpile to Kotlin `sequence { }` blocks:

```
proc count_up(n):
  let i = 0
  while i < n:
    yield i
    i = i + 1
```

Emits:

```
fun count_up(n: SageVal): Sequence<SageVal> = sequence {
  var i = S.num(0.0)
  while (S.truthy(S.lt(i, n))) {
    yield(i)
    i = S.add(i, S.num(1.0))
  }
}
```

Async/Await (Coroutines)

Async procs emit `suspend fun`; await uses `kotlinx.coroutines.runBlocking`:

```
async proc fetch():
  return 42

let result = await fetch()
```

Memory Operations on Android

`mem_alloc`/`mem_read`/`mem_write`/`mem_free` map to `java.nio.ByteBuffer`:

```
let buf = mem_alloc(256)
mem_write(buf, 0, "int", 42)
print(mem_read(buf, 0, "int")) # 42
mem_free(buf)
```

Jetpack Compose

When `<font face="mono" size="8.4">import android.compose</font>` is detected, the project generator uses Compose:

- `<font face="mono" size="8.4">@Composable</font>` Activity with Material 3
- Compose BOM, navigation-compose, ui-tooling dependencies
- `<font face="mono" size="8.4">ComponentActivity</font>` + `<font face="mono" size="8.4">setContent { }</font>` instead of programmatic views

GPU Graphics on Android

The `<font face="mono" size="8.4">lib/android/graphics.sage</font>` module provides Vulkan-style and OpenGL ES APIs:

```
import android.graphics

let gpu = GPUContext("My App")
gpu.initialize()

# Create compute buffer and run shader
let buf = gpu.create_buffer(1024, "storage")
gpu.upload(buf, [1.0, 2.0, 3.0])
gpu.dispatch_compute("shader.comp", buf, 256)
let result = gpu.download(buf)

# OpenGL ES convenience
let gl = GLESContext()
gl.initialize()
gl.clear(0.1, 0.1, 0.2, 1.0)
gl.draw_arrays("triangles", 0, 3)
gl.swap_buffers()
```

Concurrency on Android

Kotlin transpiler maps Sage concurrency to JVM primitives:

```
# Atomics → java.util.concurrent.atomic.AtomicLong
let counter = atomic_new(0)
atomic_add(counter, 1)

# Semaphores → java.util.concurrent.Semaphore
let sem = sem_new(3)
sem_wait(sem)
sem_post(sem)

# CPU info (via Runtime.getRuntime())
let cores = cpu_count()
```

Additional Mapped Builtins

The Kotlin transpiler now maps 60+ built-in functions:

- **Strings**: `<font face="mono" size="8.4">upper</font>`, `<font face="mono" size="8.4">lower</font>`, `<font face="mono" size="8.4">strip</font>`, `<font face="mono" size="8.4">split</font>`, `<font face="mono" size="8.4">join</font>`, `<font face="mono" size="8.4">replace</font>`, `<font face="mono" size="8.4">chr</font>`, `<font face="mono" size="8.4">ord</font>`
- **Paths**: `<font face="mono" size="8.4">path_join</font>`, `<font face="mono" size="8.4">path_exists</font>`, `<font face="mono" size="8.4">path_basename</font>`,

path_dirname, path_ext

- Timing: clock() (System.nanoTime)
- Hashing: hash(v) (Object.hashCode)

\newpage

Part VIe: Performance Optimization

Hybrid JIT/AOT Architecture

The self-hosted interpreter implements a profile-guided type specialization engine that runs automatically (no flags needed):

How It Works

- **Profiling Phase**: Every function call records the argument types and increments a call counter. After `HOT_THRESHOLD` (50) calls, the function is analyzed.
- **Monomorphic Detection**: If all observed calls passed the same argument types (e.g., always numbers), the function is marked as `"specialized"`.
- **Type-Feedback Interpretation**: Specialized functions always hit the number fast-path in `eval_binary` (inline arithmetic, no dispatch table lookup). This gives 100% fast-path rate vs ~70% without profiling.
- **Loop Specialization**: While-loops profile the first 8 iterations. If the body never returns break/return/continue signals, the loop switches to a `"fast mode"` that skips signal checking entirely — a ~30% speedup on tight loops.

Example

```
proc fib(n):
  if n <= 1:
    return n
  return fib(n - 1) + fib(n - 2)

# After ~50 calls, fib() is profiled as monomorphic (all-number args).
# The number fast-path in eval_binary is taken on every binary operation.
print fib(28)
```

Profiling API

```
# These are internal but accessible:
let profile = _get_profile("fib") # Get profile for a function
# profile["calls"] → 317811 (call count)
# profile["arg_types"] → ["number"]
# profile["monomorphic"] → true
# profile["specialized"] → true
```

Performance Library (`lib/perf.sage``)

The `perf` module provides reusable optimization primitives:

Signal Singletons

Eliminate dict allocation on every statement return:

```
import perf

# Instead of allocating {"kind": 0, "value": nil} every time:
let result = perf.sig_normal_nil() # Returns pre-allocated singleton
let brk = perf.sig_break()        # Zero allocation
```

Dispatch Tables

Replace if/elif chains with O(1) dict lookup:

```
import perf

let table = perf.make_dispatch_table()
perf.dispatch_register(table, "add", proc(args): return args[0] + args[1])
perf.dispatch_register(table, "sub", proc(args): return args[0] - args[1])

# O(1) dispatch instead of N comparisons:
let result = perf.dispatch_call(table, "add", [10, 20])
```

Shape Objects

Pre-shaped dict constructors for known structures:

```
import perf

let func = perf.shape_function("add", params, body, closure, false)
let cls = perf.shape_class("Dog", parent)
let inst = perf.shape_instance(cls)
let env = perf.shape_env(parent_env)
```

Fast Numeric Operations

Bypass type dispatch for known-numeric paths:

```
import perf

let sum = perf.fast_sum([1, 2, 3, 4, 5])
let total = perf.fast_add_num(a, b)
```

Flat Environment Cache

O(1) variable access bypassing scope chains in tight loops:

```
import perf

let cache = perf.flat_cache_snapshot(env)
# ... hot loop using flat_cache_get/set ...
perf.flat_cache_flush(cache, env)
```

Self-Hosted Interpreter Optimizations

The self-hosted interpreter (`src/sage/interpreter.sage`) applies these patterns:

- **Pre-allocated signal singletons:** `result_normal(nil)`, `result_break()`, `result_continue()` return cached dicts

- **Native dispatch table**: `call_native()` uses $O(1)$ dict lookup instead of 180-line if/elif chain
- **Binary op dispatch table**: `eval_binary()` uses $O(1)$ dict lookup for all operators
- **Recursion depth at call boundaries only**: `eval_expr()` no longer increments/decrements depth counter
- **Shape constructors**: functions, classes, environments built as single dict literals

Native C Interpreter Optimizations

The C interpreter (`src/c/interpreter.c`, `src/c/env.c`) applies:

- **Cached name length in EnvNode**: `name_length` field avoids `strlen` during lookup
- **memcmp with length pre-check**: `env_get`, `env_define`, `env_assign` check `name_length == length` before `memcmp`
- **Inlined eval_expr**: a stack-proximity guard (`stack_danger()`) checks real stack usage at both `interpret()` and per-expression boundaries, so deep expression nesting fails with a catchable exception before the OS stack is exhausted (v4.1.16)
- **For-loop slot caching**: loop variable node pointer cached after first `env_define`, subsequent iterations write directly
- **String pointer equality**: `values_equal()` checks `AS_STRING(a) == AS_STRING(b)` before `strcmp`

Algorithmic Optimizations (v4.1.5)

Recent updates have transitioned key library operations from interpreted loops to native C implementations or more efficient algorithms:

- **Dictionary Size**: `dicts.size(d)` now uses the native `len(d)` builtin, achieving $O(1)$ complexity (~590x speedup).

- **Array Uniqueness**: `arrays.unique(arr)` now uses a dictionary for lookups, achieving $O(N)$ average-case complexity (previously $O(N^2)$).

- **Array Reversal**: `arrays.reverse(arr)` now uses a native C implementation (`array_reverse`), achieving ~105x speedup.

- **String/Value Repeating**: `strings.repeat` and `utils.repeat_value` now use

binary exponentiation and `array_extend`, achieving $O(\log n)$ efficiency (~13x speedup).

- **Array Partitioning**: `arrays.take` and `arrays.drop` now use native `slice`, achieving ~88x speedup.

Cross-Backend Benchmarks

Run the 8-workload benchmark across all backends:

```
bash benchmarks/run_backend_compare.sh
python3 scripts/generate_backend_chart.py
```

Workloads: fibonacci, loop sum, array ops, string concat, dict ops, prime sieve, nested loops, LCG hash.

\newpage

Part VIe: Default Runtime and Execution Modes

JIT+AOT Hybrid Default

As of v4.1.5, Sage's default runtime is `auto` — JIT profiling mode on hosted platforms, AST interpreter on bare-metal:

```
| | | E | n | v | i | r | o | n | m | e | n | t | | | | A | u | t | o | | R | e | s | o | l | v | e | s | | | | | | | | | | | | | |
| T | o | | | | W | h | y | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | |
| | | D | e | s | k | t | o | p | / | S | e | r | v | e | r | | | | J | I | T | | p | r | o | f | i | l | i
| n | g | | | | F | u | l | l | | O | S | , | | t | y | p | e | | f | e | e | d | b | a | c | k | | c |
o | l | l | e | c | t | i | o | n | | | |
| | | A | n | d | r | o | i | d | | | | J | I | T | | p | r | o | f | i | l | i | n | g | | | | J | V
| M | | + | | c | o | r | o | u | t | i | n | e | s | | a | v | a | i | l | a | b | l | e | | | | | | |
| | | | | | | | | | |
| | | B | a | r | e | - | m | e | t | a | l | / | P | i | c | o | | | | A | S | T | | i | n | t | e | r | p
| r | e | t | e | r | | | | N | o | | f | l | o | r | k | / | s | y | s | t | e | m | , | | s | a | f | e | |
f | a | l | l | b | a | c | k | | | |
| | | K | e | r | n | e | l | | d | e | v | | | | S | a | g | e | M | e | t | a | l | | V | M | | | |
| F | r | e | e | s | t | a | n | d | i | n | g | , | | z | e | r | o | | a | l | l | o | c | a | t | i | o | n |
| | | | | | | | | | |
```

Override with `--runtime ast`, `--runtime bytecode`, `--runtime jit`, or `--runtime aot`.

Runtime Pragma Decorators

Control JIT/AOT behavior per-function:

```
@nojit
proc low_level_driver():
    # Skips JIT profiling for this function
    mem_write(ptr, 0, "int", 42)

@noaot
proc dynamic_dispatch():
    # Skips AOT compilation
    ...

@nopprofile
proc realtime_handler():
    # Skips all profiling overhead
    ...
```

\newpage

Part VIg: SageMetal VM — Bare-Metal Bytecode Interpreter

The SageMetal VM is a freestanding bytecode interpreter that runs without an OS, libc, or dynamic memory allocation. It is designed for kernels, bootloaders, embedded systems, and OS development.

Architecture

All storage is static — no malloc, no heap fragmentation:

- **Value Stack**: 512 slots (configurable via `METAL_STACK_SIZE`)
- **Constant Pool**: 1024 entries
- **String Pool**: 32KB bump allocator (interned, deduplicated)
- **Heap**: 64KB bump allocator for general use
- **Scope Chain**: 64 levels deep, 32 variables per scope
- **Array Pool**: Fixed-capacity arrays (256 elements each)

MetalValue Type

Compact 16-byte tagged union:

```
MV_NIL    - null value
MV_NUM    - IEEE 754 double
MV_BOOL   - 0 or 1
MV_STR    - index into string pool
MV_ARR    - index into array pool
MV_DICT   - index into dict pool
MV_FN     - index into function table
MV_PTR    - raw pointer (for MMIO, DMA)
```

Host I/O Callbacks

The kernel or bootloader sets these before running the VM:

```
MetalVM vm;
metal_vm_init(&vm);
vm.write_char = serial_putchar; // Console output
vm.read_char = serial_getchar;  // Console input
vm.write_port = x86_outb;       // Port I/O
vm.read_port = x86_inb;         // Port I/O
vm.map_mmio = identity_map;     // MMIO mapping

metal_vm_load(&vm, bytecode, length);
metal_vm_run(&vm); // Execute until halt
```

Single-Step Mode

For cooperative multitasking in kernels:

```
while (metal_vm_step(&vm)) {  
    // Check interrupts, handle timer ticks, etc.  
    if (timer_interrupt_pending) handle_timer();  
}
```

Building

```
make metal-vm    # Produces obj/metal_vm.o (freestanding)
```

Link with your kernel:

```
gcc -ffreestanding -nostdlib -o kernel.elf \  
    boot.o kernel.o obj/metal_vm.o -lgcc
```

\newpage

Part VIh: Metal Standard Library

The `lib/metal/` modules provide bare-metal primitives for kernel and embedded development.

metal.core

Core primitives: serial I/O, port I/O, MMIO, CPU control, memory.

```
import metal.core

# Console
core.puts("Hello from kernel!")

# Port I/O (x86)
core.outb(0x3F8, 65)    # Send 'A' to COM1
let val = core.inb(0x60) # Read keyboard scancode

# MMIO
core.mmio_write32(0xB8000, 0x0F41) # Write 'A' to VGA

# CPU control & Memory barriers
core.cli()    # Disable interrupts
core.sti()    # Enable interrupts
core.hlt()    # Halt until next interrupt
core.dmb()    # ARM Data Memory Barrier
core.dsb()    # ARM Data Synchronization Barrier
core.isb()    # ARM Instruction Synchronization Barrier
core.fence()  # RISC-V Memory Fence

# Multicore & Synchronization
let cid = core.cpu_id()
core.critical_section_enter()
# ... atomic operation ...
core.critical_section_exit()

core.spin_lock(0x1000)
# ... protected section ...
core.spin_unlock(0x1000)

# Bump allocator
core.heap_init(0x100000, 65536)
let ptr = core.heap_alloc(256)
```

metal.serial

UART drivers for x86 (NS16550A) and ARM (PL011):

```

| | | F | u | n | c | t | i | o | n | | | | D | e | s | c | r | i | p | t | i | o | n | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | u | a | r | t | _ | i | n | i | t | ( | p | o | r | t | , | | b | a | u | d | ) | ` | | | | I | n
| | t | i | a | l | l | i | z | e | | N | S | 1 | 6 | 5 | 5 | 0 | A | | C | 0 | M | | p | o | r | t | | a | t | | | | | | | | | | |
| | s | p | e | c | i | f | i | e | d | | b | a | u | d | | r | a | t | e | . | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | u | a | r | t | _ | i | n | i | t | _ | e | x | t | ( | p | o | r | t | , | | b | a | u | d | , | | b
| | t | s | , | | s | t | o | p | , | | p | a | r | i | t | y | ) | ` | | | | E | x | t | e | n | d | e | | | | | | | | | | | |
| | d | | C | 0 | M | | p | o | r | t | | i | n | i | t | i | a | l | l | i | z | a | t | i | o | n | . | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | u | a | r | t | _ | s | e | n | d | ( | p | o | r | t | , | | b | y | t | e | ) | ` | | | | T | r
| | a | n | s | m | i | t | | s | i | n | g | l | e | | b | y | t | e | | ( | b | l | l | o | c | k | i | n | g | ) | | | | | | | | | | | |
| | . | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | u | a | r | t | _ | r | e | c | v | ( | p | o | r | t | ) | ` | | | | R | e | c | e | i | v | e |
| | s | i | n | g | l | e | | b | y | t | e | | ( | b | l | l | o | c | k | i | n | g | ) | . | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | u | a | r | t | _ | p | u | t | s | ( | p | o | r | t | , | | s | ) | ` | | | | T | r | a | n | s
| | m | i | t | | s | t | r | i | n | g | | o | v | e | r | | C | 0 | M | | p | o | r | t | . | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | u | a | r | t | _ | r | e | a | d | _ | t | i | m | e | o | u | t | ( | p | o | r | t | , | | t | i | m
| | e | o | u | t | _ | m | s | ) | ` | | | | R | e | c | e | i | v | e | | s | i | n | g | l | e | | b | y |
| | t | e | | w | i | t | h | | t | i | m | e | o | u | t | | i | n | | m | i | l | l | i | s | e | c | o | n | d
| | s | | ( | r | e | t | u | r | n | s | | i | n | | l | | o | n | | t | i | m | e | o | u | t | ) | . | | | | | |
| | | ` | u | a | r | t | _ | r | e | a | d | l | i | n | e | ( | p | o | r | t | ) | ` | | | | R | e | a | d
| | l | i | n | e | | ( | u | n | t | i | l | | n | e | w | l | i | n | e | / | C | R | ) | | w | i | t | h |
| | t | e | r | m | i | n | a | l | | e | c | h | o | | a | n | d | | b | a | c | k | s | p | a | c | e | | s
| | u | p | p | o | r | t | . | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | u | a | r | t | _ | f | l | u | s | h | _ | r | x | ( | p | o | r | t | ) | ` | | | | F | l | u | s
| | h | | p | e | e | n | d | i | n | g | | b | y | t | e | s | | i | n | | R | X | | b | u | f | f | e | r | . | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | p | l | 0 | 1 | 1 | _ | i | n | i | t | ( | b | a | s | e | ) | ` | | | | I | n | i | t | i | a | l
| | i | z | e | | A | R | M | | P | L | 0 | 1 | 1 | | U | A | R | T | | a | t | | b | a | s | e | | a | d
| | d | r | e | s | s | . | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | p | l | 0 | 1 | 1 | _ | s | e | n | d | ( | b | a | s | e | , | | b | y | t | e | ) | ` | | | | T
| | r | a | n | s | m | i | t | | s | i | n | g | l | e | | b | y | t | e | | v | i | a | | P | L | 0 | 1 | 1 | | | | | | | | | | | | |
| | . | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | p | l | 0 | 1 | 1 | _ | r | e | c | v | ( | b | a | s | e | ) | ` | | | | R | e | c | e | i | v | e
| | | s | i | n | g | l | e | | b | y | t | e | | v | i | a | | P | L | 0 | 1 | 1 | | ( | b | l | l | o | c | k | | | | | | | | | |
| | i | n | g | ) | . | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | p | l | 0 | 1 | 1 | _ | p | u | t | s | ( | b | a | s | e | , | | s | ) | ` | | | | T | r | a | n
| | s | m | i | t | | s | t | r | i | n | g | | v | i | a | | P | L | 0 | 1 | 1 | . | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | ` | p | l | 0 | 1 | 1 | _ | r | e | a | d | _ | t | i | m | e | o | u | t | ( | b | a | s | e | , | | t | i
| | m | e | o | u | t | _ | m | s | ) | ` | | | | R | e | c | e | i | v | e | | s | i | n | g | l | e | | b |

```

Interrupt management and PIC control:

IRQ API

[illegible]

Hardware timer with tick counting and sleep:

Timer API

Page 83

```
import metal.gpio

gpio.gpio_init(0x40000000, 32) # Init 32 pins at MMIO base
gpio.pin_mode(25, gpio.PIN_OUTPUT) # LED pin
gpio.led_blink(25, 5, 250) # Blink 5 times, 250ms interval

# Batch operations
gpio.pin_set_mask(0b1011) # Set pins 0, 1, 3 HIGH
gpio.pin_clear_mask(0b1011) # Set pins 0, 1, 3 LOW
gpio.pin_write_masked(0b11, 0b10) # Set pin 1 HIGH, pin 0 LOW
```

Constants:

- **Modes**: `<font face="mono" size="8.4">PIN_INPUT</font>`, `<font face="mono" size="8.4">PIN_OUTPUT</font>`, `<font face="mono" size="8.4">PIN_ALT</font>`, `<font face="mono" size="8.4">PIN_ANALOG</font>`
- **Levels**: `<font face="mono" size="8.4">PIN_LOW</font>`, `<font face="mono" size="8.4">PIN_HIGH</font>`
- **Pulls**: `<font face="mono" size="8.4">PULL_NONE</font>`, `<font face="mono" size="8.4">PULL_UP</font>`, `<font face="mono" size="8.4">PULL_DOWN</font>`
- **Interrupts**: `<font face="mono" size="8.4">INT_DISABLED</font>`, `<font face="mono" size="8.4">INT_RISING</font>`, `<font face="mono" size="8.4">INT_FALLING</font>`, `<font face="mono" size="8.4">INT_BOTH</font>`, `<font face="mono" size="8.4">INT_LOW</font>`, `<font face="mono" size="8.4">INT_HIGH</font>`

```
| | | F | u | n | c | t | i | o | n | | | | D | e | s | c | r | i | p | t | i | o | n | | | | | | | |
| | | ` | p | i | n | _ | m | o | d | e | ( | p | , | | m | ) | ` | | | | S | e | t | | m | o | d | e |
| ( | I | N | P | U | T | , | | O | U | T | P | U | T | , | | A | L | T | , | | A | N | A | L | O | G | ) | . |
| | | | ` | p | i | n | _ | g | e | t | _ | m | o | d | e | ( | p | ) | ` | | | | G | e | t | | c | u | r |
| e | n | t | | p | i | n | | m | o | d | e | . | | | | | | | | | | | | | |
| | | | ` | p | i | n | _ | p | u | l | l | ( | p | , | | c | f | g | ) | ` | | | | S | e | t | | p | u | l |
| l | | c | o | n | f | i | g | u | r | a | t | i | o | n | | ( | N | O | N | E | , | | U | P | , | | D | O |
W | N | ) | . | | | | | | | | | | | | | |
| | | | ` | p | i | n | _ | g | e | t | _ | p | u | l | l | ( | p | ) | ` | | | | G | e | t | | c | u | r | r |
| e | n | t | | p | u | l | l | | c | o | n | f | i | g | u | r | a | t | i | o | n | . | | | | | | | |
| | | | | | | | | | | | | |
| | | | ` | p | i | n | _ | s | e | t | _ | i | n | t | e | r | r | u | p | t | ( | p | , | | m | ) | ` | | | |
| S | e | t | | i | n | t | e | r | r | u | p | t | | t | r | i | g | g | e | r | | ( | R | I | S | I | N | G |
| | F | A | L | L | I | N | G | , | | B | O | T | H | , | | e | l | t | c | ) | . | | |
| | | | ` | p | i | n | _ | g | e | t | _ | i | n | t | e | r | r | u | p | t | ( | p | ) | ` | | | | G | e | t |
| | c | u | r | r | e | n | t | | i | n | t | e | r | r | u | p | t | | m | o | d | e | . | | | | | | | |
| | | | | | | | | | | | | |
| | | | ` | p | i | n | _ | r | e | g | i | s | t | e | r | _ | h | a | n | d | l | e | r | ( | p | , | | f | ) | `
| | | | R | e | g | i | s | t | e | r | | c | a | l | l | b | a | c | k | | ` | f | ` | | f | l | o | r | | p |
i | n | | ` | p | ` | . | | | | | | | | | |
| | | | ` | p | i | n | _ | e | n | a | b | l | e | _ | i | n | t | e | r | r | u | p | t | ( | p | ) | ` | | | | |
| E | n | a | b | l | e | | i | n | t | e | r | r | u | p | t | | f | l | o | r | | p | i | n | | ` | p | ` | . |
| | | | | | | | | | | | | |
| | | | ` | p | i | n | _ | d | i | s | a | b | l | e | _ | i | n | t | e | r | r | u | p | t | ( | p | ) | ` | | | |
| D | i | s | a | b | l | e | | i | n | t | e | r | r | u | p | t | | f | l | o | r | | p | i | n | | ` | p | ` |
. | | | | | | | | | |
| | | | ` | g | p | i | o | _ | d | i | s | p | a | t | c | h | ( | p | ) | ` | | | | M | a | n | u | a | l | | |
| y | | d | i | s | p | a | t | c | h | | i | n | t | e | r | r | u | p | t | | f | l | o | r | | p | i | n | |
| ` | p | ` | . | | | | | | | | | |
| | | | ` | p | i | n | _ | d | e | b | o | u | n | c | e | ( | p | , | | s | , | | n | , | | d | ) | ` | | | |
| D | e | b | o | u | n | c | e | | p | i | n | | ` | p | ` | | t | o | | s | t | a | t | e | | ` | s | ` |
```


Combining the Metal library into a minimal "Hello World" kernel with keyboard input:

```
import metal.core as core
import metal.serial as serial
import metal.irq as irq
import metal.timer as timer

proc kernel_main():
  # 1. Initialize hardware
  serial.uart_init(serial.COM1, 115200)
  timer.pit_init(100) # 100Hz
  irq.pic_remap(32, 40)

  serial.uart_puts(serial.COM1, "SageMetal Kernel v1.0 Booting...\n")

  # 2. Setup keyboard interrupt (vector 33 for IRQ 1)
  irq.register_handler(33, proc(vector):
    let scancode = core.inb(0x60)
    serial.uart_puts(serial.COM1, "Key pressed: " + str(scancode) + "\n")
    irq.pic_eoi(1)
  )
  irq.pic_unmask(1) # Keyboard

  # 3. Main Loop
  serial.uart_puts(serial.COM1, "Kernel ready. Type something...\n")
  core.sti() # Enable interrupts

  var last_print = 0
  while true:
    let now = timer.ticks()
    if now - last_print > 500: # Every 5 seconds at 100Hz
      serial.uart_puts(serial.COM1, "[Heartbeat] Uptime: " + str(now/100) + "s\n")
      last_print = now
    end
    core.hlt() # Wait for next interrupt
  end
end
```

\newpage

Part VII: Blockchain and Distributed Ledger Technology

The `lib/blockchain/` library provides a pure SageLang implementation of an enterprise-grade L1 blockchain.

Architecture

The system is highly modular, with core components separating concerns:

- **Blockchain Core** (`blockchain.blockchain`): Manages the main ledger state, mempool, and consensus coordination.
- **Ledger Storage** (`blockchain.db`): Provides high-performance, disk-backed storage for blocks, transactions, and world state.
- **Consensus Engines** (`blockchain.consensus.*`): Pluggable architecture supporting:
 - **Proof-of-Work (PoW)**: Standard mining difficulty adjustment.
 - **Proof-of-Authority (PoA)**: Validator-based consensus with automatic slashing for equivocation.
- **World State Trie** (`blockchain.merkle`): A persistent Merkle-Radix Trie for global account balances and contract state, ensuring cryptographically verifiable `state_root`s.
- **Network Layer** (`blockchain.net`, `blockchain.rpc`): Implements P2P node discovery, Initial Block Download (IBD), fork resolution, and a standard JSON-RPC 2.0 API for dApp integration.

Key Features

- **Smart Contracts & NFTs**: Includes a native VM with gas metering and support for inter-contract calls. The **SNFT-721 standard** provides full support for non-fungible tokens.
- **Wallet & Security**: Supports BIP-39 style deterministic HD wallets for address generation and transaction signing. If `libsage_crypto.so` (Ed25519) is available via FFI, it provides high-performance hardware-accelerated signatures.
- **Priority Fee Market**: Mempool implementation that prioritizes transactions based on `gas_price`, dynamically incentivizing miners.
- **Orbit Dynamic Mining**: A dynamic mining rate model that adjusts rewards based on network adoption, total supply, and node reliability.
- **Staking System**: Logic for ORBIT token staking via system smart contracts to provide passive rewards (~5% APR).

Usage Example

The following example demonstrates how to set up a blockchain with PoW, create a wallet, add and sign a transaction, and mine a block:

```
import blockchain.blockchain as bc_mod
import blockchain.wallet as wallet_mod
import blockchain.consensus.pow as pow_mod
import blockchain.transaction as tx_mod

# Initialize a blockchain with PoW consensus
let consensus = pow_mod.PowConsensus(nil, 2)
let coin = bc_mod.Blockchain(consensus, "./sagechain_db")
consensus.blockchain = coin

# Generate a new wallet
let wallet = wallet_mod.Wallet(nil)

# Create and sign a transaction
let tx = coin.add_transaction(wallet.get_address(), "recipient", 100)
wallet.sign_transaction(tx)
coin.add_signed_transaction(tx)

# Mine the pending transactions
coin.mine_pending_transactions("miner-address")

# Retrieve balance
print coin.get_balance(wallet.get_address())
```

For advanced use cases, including P2P networking and validator configuration, refer to the examples in `examples/blockchain_*.sage`.

\newpage

Part VIII: Discord Bot Library

SageLang now includes a library for building Discord bots, designed to mirror the familiarity of Python's `discord` and `discord.ext` libraries.

Getting Started

To create a Discord bot, import the `discord.client` module:

```
import discord.client

proc on_ready(data):
    print("Bot is ready!")
end

# Intent 32767 = all intents
let bot = discord.client.Client("YOUR_TOKEN", 32767)
bot.on("READY", on_ready)
bot.run()
```

REST API Support

The library includes an HTTP client for interacting with Discord's REST API. You can send messages directly:

```
bot.send_message(CHANNEL_ID, "Hello, world!")
```

Features

- **Gateway API**: Full support for real-time event handling via WebSockets.
- **REST API**: Wrapper for sending messages, managing channels, and more.
- **Event System**: Easy-to-use `on()` listener pattern.

\newpage

Part VIIIb: Machine Learning and AI

Large Language Models (LLM)

SageLang provides a comprehensive LLM toolkit in `lib/llm`.

TurboQuant (ICLR 2026)

TurboQuant provides near-optimal vector quantization for KV cache and weights.

- **3-bit compression**: 6x memory reduction with zero accuracy loss.
- **Two-stage**: PolarQuant (MSE optimal) + QJL (Johnson-Lindenstrauss residual correction).
- **Unbiased**: Inner product estimation remains unbiased.

```
import llm.turboquant as tq

let vec = tq.vec_random(128)
let quantized = tq.quantize(vec, 3)
let reconstructed = tq.dequantize(quantized)
```

AutoResearch Agent

An autonomous research agent inspired by Karpathy's work.

- **Ratchet loop**: Propose → Train → Evaluate → Accept/Reject.
- **Multi-agent**: Collaboration between researcher, programmer, and critic agents.
- **Journal**: Keeps a detailed research log of all experiments.

Self-Evolving Architectures

`lib/llm/evolve.sage` enables models to grow their architecture over time.

- **Progressive growth**: Identity-init for depth, weight padding for width.
- **Plateau detection**: Automatically grows when training loss plateaus.

\newpage

Part IX: Appendices

\newpage

Core Functions

Array Functions

Page 92

[illegible][illegible]

			F	u	n	c	t		i	o	n									D	e	s	c	r	i	p	t		i	o	n			
				n	e	x	t		(	g	e	n	)								A	d	v	a	n	c	e			g	e	n	e	
	r	a		t	o	r		a	n	d		r	e	t		u	r	n		n	e	x	t			v	a		l		u	e		

Page 93

		F	u	n	c	t	i	o	n											D											
	e	s	c	r	i	p	t	i	o	n																					
			\	p	a	t	h	_	j	o	i	n	(	a	,		b	)	\	J											
	o	i	n		t	w	o		p	a	t	h		c	l	o	m	p	l	o											
			\	p	a	t	h	_	d	i	r	n	a	m	e	(	p	a	t	h	)	\	D								
	i	r	e	c	t	o	r	y		p	l	o	r	t	i	o	n		o	f		p	a	t	h						
			\	p	a	t	h	_	b	a	s	e	n	a	m	e	(	p	a	t	h	)	\								F
	i	l	e	n	a	m	e		p	l	o	r	t	i	o	n		o	f		p	a	t	h							
			\	p	a	t	h	_	e	x	t	(	p	a	t	h	)	\													F
	i	l	e		e	x	t	e	n	s	i	o	n																		
			\	p	a	t	h	_	e	x	i	s	t	s	(	p	a	t	h	)	\										C
	h	e	c	k		i	f		p	a	t	h		e	x	i	s	t	s												
			\	p	a	t	h	_	i	s	_	d	i	r	(	p	a	t	h	)	\										C
	h	e	c	k		i	f		p	a	t	h		i	s		a		d	i	r	e	c	t	o	r	y				
			\	p	a	t	h	_	i	s	_	f	i	l	e	(	p	a	t	h	)	\									C
	h	e	c	k		i	f		p	a	t	h		i	s		a		r	e	g	u	l	a	r		f	i	l		
e			\	p	a	t	h	_	m	k	p	a	t	h	(	p	a	t	h	)	\										C
	r	e	a	t	e		f	u	l	l		d	i	r	e	c	t	o	r	y		p	a	t	h						

[illegible][illegible]

```

| e | s | c | r | i | p | t | i | o | n | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | b | u | i | l | d | _ | q | u | a | d | _ | v | e | r | t | s | ( | a | r | r | ) | \ | | | | | | | | B
| a | t | c | h | | g | e | n | | e | r | a | t | e | | v | e | r | t | i | c | e | s | | f | o | r | | q | u |
a | d | s | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | b | u | i | l | d | _ | l | i | n | e | _ | q | u | a | d | s | ( | a | r | r | , | | t | , | | . | .
| . | ) | \ | | | | | C | o | n | v | e | r | t | | l | i | n | e | s | | t | o | | t | h | i | c | k | |
q | u | a | d | s | | | | | | | | | | | | | | | | | | | | | | | | | | |

```

Bytes Functions

```

| | | F | u | n | c | t | i | o | n | | | | | | | | | | | | | | | | | | | | | | |
| | | D | e | s | c | r | i | p | t | i | o | n | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | b | y | t | e | s | ( | n | ) | \ | | | | | | | | | | | | | | | | |
n | | | C | r | e | a | t | e | | b | y | t | e | | b | u | f | f | e | r | | o | f | | s | i | z | e | |
| | | | | b | y | t | e | s | _ | l | e | n | ( | b | ) | \ | | | | | | | | | | | | | | | |
| | | | | L | e | n | g | t | h | | o | f | | b | y | t | e | | b | u | f | f | e | r | | | | | | | |
| | | | | b | y | t | e | s | _ | g | e | t | ( | b | , | | i | ) | \ | | | | | | | | | | | |
| | | | | G | e | t | | b | y | t | e | | a | t | | i | n | d | e | x | | | | | | | | | | | |
| | | | | b | y | t | e | s | _ | s | e | t | ( | b | , | | i | , | | v | ) | \ | | | | | | | | | |
| | | | | S | e | t | | b | y | t | e | | a | t | | i |n | d | e | x | | | | | | | | | | | |
| | | | | b | y | t | e | s | _ | t | o | _ | s | t | r | i | n | g | ( | b | ) | \ | | | | | | | | | |
| | | | | C | o | n | v | e | r | t | | b | y | t | e | s | | t | o | | s | t | r | i | n | g | | | | |
| | | | | b | y | t | e | s | _ | s | l | i | c | e | ( | b | , | | s | t | a | r | t | , | | e | n | d | ) | \
| | | | | S | l | i | c | e | | b | y | t | e | | b | u | f | f | e | r | | | | | | | | | | | |
| | | | | b | y | t | e | s | _ | p | u | s | h | ( | b | , | | v | ) | \ | | | | | | | | | | | |
| | | | | A | p | p | e | n | d | | b | y | t | e | | t | o | | b | u | f | f | e | r | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

```

Memory Functions

```

| | | F | u | n | c | t | i | o | n | | | | | | | | | | | | | | | | | | | | | | |
| | | r | i | p | t | i | o | n | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | m | e | m | _ | a | l | l | o | c | ( | s | i | z | e | ) | \ | | | | | | | | | |
| c | a | t | e | | r | a | w | | m | e | m | o | r | y | | | | | | | | | | | | | | | |
| | | | | m | e | m | _ | f | r | e | e | ( | p | t | r | ) | \ | | | | | | | | | |
| | a | l | l | o | c | a | t | e | d | | m | e | m | o | r | y | | | | | | | | | | | | | |
| | | | | m | e | m | _ | r | e | a | d | ( | p | t | r | , | | o | f | f | , | | t | y | p | e | ) | \ | |
| | | | | R | e | a | d | | v | a | l | l | u | e | | a | t | | p | o | i | n | t | e | r | | o | f | f | s | e |
t | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | m | e | m | _ | w | r | i | t | e | ( | p | t | r | , | | o | f | f | , | | t | y | p | e | , | | v
| a | l | ) | \ | | | | | | W | r | i | t | e | | v | a | l | l | u | e | | a | t | | p | o | i | n | t | e | r |
| o | f | f | s | e | t | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | m | e | m | _ | s | i | z | e | ( | p | t | r | ) | \ | | | | | | | | | |
| | o | f | | a | l | l | o | c | a | t | i | o | n | | | | | | | | | | | | | | | |
| | | | | a | d | d | r | e | s | s | o | f | ( | v | a | l | ) | \ | | | | | | | | | |
| m | e | m | o | r | y | | a | d | d | r | e | s | s | | o | f | | a | | v | a | l | l | u | e | | | | |
| | | | | a | d | d | r | e | s | s | o | f | _ | r | a | w | ( | v | a | l | ) | \ | | | | | | | | | |
| | r | a | w | | m | e | m | o | r | y | | a |d |d |r |e |s |s | | o |f | | a | | v | a |l |l |u |e |
| | | | | s | i | z | e | o | f | ( | t | y | p | e | ) | \ | | | | | | | | | |
| | o | f | | a | | t | y | p | e | | i |n | | b | y | t | e | s | | | | | | | | | | | |
| | | | | p | t | r | _ | a | d | d | ( | p | t | r | , | | o | f | f | ) | \ | | | | | | | | | |
| | t | e | r | | a | r | i | t | h | m | e | t | i | c | | | | | | | | | | | | | | | |
| | | | | p | t | r | _ | t | o | _ | i | n | t | ( | p | t | r | ) | \ | | | | | | | | | |
| | e | r | t | | p | o | i | n | t | e | r | | t | o | | i |n |t |e |g |e |r | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

```

FFI Functions

```

| | | F | u | n | c | t | i | o | n | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | D | e | s | c | r | i | p | t | i | o | n | | | | | | | | | |

```

```

| | | f | f | i | _ | o | p | e | n | ( | l | l | i | b | _ | p | l | a | t | h | ) | ` | | | | | | | | | |
| | | | | | | O | p | e | n | | s | h | a | r | e | d | | l | l | i | b | r | a | r | y | | | | | |
| | | f | f | i | _ | c | l | o | s | e | ( | h | a | n | d | l | e | ) | ` | | | | | | | | |
| | | | | | | C | l | o | s | e | |s | h | a | r | e | d | | l | l | i | b | r | a | r | y | | | | | |
| | | f | f | i | _ | s | y | m | ( | h | a | n | d | l | e | , | | n | a | m | e | ) | ` | | | | | | |
| | | | | | | L | o | o | k | |u | p | |s | y | m | b | o | l | | | | | | | | | | |
| | | f | f | i | _ | c | a | l | l | ( | s | y | m | , | | a | r | g | s | , | | r | e | t | _ | t | y | p |
| e | ) | ` | | | | C | a | l | l | |f | o | r | e | i | g | n | |f | u | n | c | t | i | o | n | | | |
| | | | | | | | |

```

Struct Interop Functions

```

| | | F | u | n | c | t | i | o | n | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | D | e | s | c | r | i | p | t | i | o | n | | | | | | | | | | |
| | | s | t | r | u | c | t | _ | d | e | f | ( | f | i | e | l | d | s | ) | ` | | | | | | | | |
| | | | | | | | | D | e | f | i | n | e | |a | | C | |s | t | r | u | c | t | |l |a |y |
o | u | t | | | | | | | |
| | | s | t | r | u | c | t | _ | n | e | w | ( | d | e | f | ) | ` | | | | | | | | | | | |
| | | | | | | | | A | l | l | o | c | a | t | e | |a | |n |e |w | |s | t | r | u | c | t |
| | | | | | | | |
| | | s | t | r | u | c | t | _ | g | e | t | ( | p | t | r | , | | d | e | f | , | | f | i | e | l | d | )
| ` | | | | | | | | | R | e | a | d | |s | t | r | u | c | t | |f | i | e | l | d | | | | |
| | | | | | | | |
| | | s | t | r | u | c | t | _ | s | e | t | ( | p | t | r | , | | d | e | f | , | | f | i | e | l | d | ,
| |v |a |l | ) | ` | | | | W | r | i | t | e | |s | t | r | u | c | t | |f | i | e | l | d | | | |
| | | | | | | | |
| | | s | t | r | u | c | t | _ | s | i | z | e | ( | d | e | f | ) | ` | | | | | | | | |
| | | | | | | | | S | i | z | e | |o |f | |s | t | r | u | c | t | |i |n | |b |y |t |
e | s | | | | | | | |

```

Assembly Functions

```

| | | F | u | n | c | t | i | o | n | | | | | | | | | | | | | | | | | | | | | D | e | s | c
r | i | p | t | i | o | n | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | |
| | | a | s | m | _ | e | x | e | c | ( | c | o | d | e | ) | ` | | | | | | | | | E | x | e | c
u | t | e | |i |n | |l |i |n |e | |a |s |s |e |m |b |l |y | | | | | | | | |
| | | | | | | | |
| | | a | s | m | _ | c | o | m | p | i | l | l | e | ( | c | o | d | e | ) | ` | | | | | | | | | C | o | m | p
i | l | l | e | |a |s |s |e |m |b |l |y | |t |o | |m |a |c |h |i |n |e | |c |o |d |e | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | a | s | m | _ | a | r | c | h | ( | ) | ` | | | | | | | | | | | | | | | G | e | t |
| c | u | r | r | e | n | t | |a |r |c |h |i |t |e |c |t |u |r |e | |s |t |r |i |n |g | | |
| | | | | | | | |

```

System & SMP Functions

```

| | | F | u | n | c | t | i | o | n | | | | | | | | | | | | | | | | | | | | | D
e | s | c | r | i | p | t | i | o | n | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | |
| | | c | p | u | _ | c | o | u | n | t | ( | ) | ` | | | | | | | | | | | | | | | N
u | m | b | e | r | |o |f | |l |o |g |i |c |a |l | |C |P |U | |c |o |r |e |s | | | |
| | | | | | | | |
| | | c | p | u | _ | p | h | y |s |i |c |a |l | _ |c |o |r |e |s | ( | ) | ` | | | | | | | | | N
u | m | b | e | r | |o |f | |p |h |y |s |i |c |a |l | |C |P |U | |c |o |r |e |s | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | c | p | u | _ | h | a |s | _ |h |y |p |e |r |t |h |r |e |a |d |i |n |g | ( | ) | ` | | |
| |T |r |u |e | |i |f | |h |y |p |e |r |t |h |r |e |a |d |i |n |g | |i |s | |e |n |
a |b |l |e |d | | | | |
| | | t |h |r |e |a |d | _ |s |e |t | _ |a |f |f |i |n |i |t |y | ( |m | ) | ` | | | | | P
i |n | |c |u |r |r |e |n |t | |t |h |r |e |a |d | |t |o | |c |o |r |e | |m |a |s |
k | | | | | | | |
| | | t |h |r |e |a |d | _ |g |e |t | _ |c |o |r |e | ( | ) | ` | | | | | | | | | G
e |t | |I |D | |o |f | |c |u |r |r |e |n |t |l |y | |e |x |e |c |u |t |i |n |g | |
c |o |r |e | | | | | | |

```

Atomic Operations

```

| | | F | u | n | c | t | i | o | n | | | | | | | | | | | | | | | | | | | | | D

```

```

| e | s | c | r | i | p | t | i | o | n | | | | | | | | | | | | | | | | | | | | | |
| | | \ | a | t | o | m | i | c | _ | n | e | w | ( | v | a | l | ) | \ | | | | | | | | | | | | | | | C
| r | e | a | t | e | | n | e | w | | a | t | o | m | i | c | | v | a | r | i | a | b | l | e | | | | | | | |
| | | \ | a | t | o | m | i | c | _ | l | o | a | d | ( | a | t | o | m | ) | \ | | | | | | | | | | | | | | T
| h | r | e | a | d | - | s | a | f | e | | l | o | a | d | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | \ | a | t | o | m | i | c | _ | s | t | o | r | e | ( | a | t | o | m | , | | v | ) | \ | | | | | | | | | T
| h | r | e | a | d | - | s | a | f | e | | s | t | o | r | e | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | \ | a | t | o | m | i | c | _ | a | d | d | ( | a | t | o | m | , | | v | ) | \ | | | | | | | | | | | | | A
| t | o | m | i | c | | f | e | t | c | h | - | a | n | d | - | a | d | d | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | \ | a | t | o | m | i | c | _ | c | a | s | ( | a | , | | e | , | | d | ) | \ | | | | | | | | | | | | | A
| t | o | m | i | c | | c | o | m | p | a | r | e | - | a | n | d | - | s | w | a | p | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | \ | a | t | o | m | i | c | _ | e | x | c | h | a | n | g | e | ( | a | , | | v | ) | \ | | | | | | | | | A
| t | o | m | i | c | | e | x | c | h | a | n | g | e | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

```

\newpage

Appendix B: CLI Reference

Usage: sage [options] [file.sage]

Running:

(no args)	Interactive REPL
file.sage	Run script in interpreter
--repl	Interactive REPL (explicit)

Compilation:

--compile-to-lily FILE -o OUT	Emit Lily source code
--compile-from-lily FILE -o OUT	Compile Lily to Sage
--emit-c FILE -o OUT	Emit C source code
--compile FILE -o OUT	Compile to binary via C backend
--emit-llvm FILE -o OUT	Emit LLVM IR text
--compile-llvm FILE -o OUT	Compile to binary via LLVM (requires clang)
--emit-asm FILE -o OUT	Emit assembly source
--compile-native FILE -o OUT	Compile to native binary
--compile-bare FILE -o OUT	Bare-metal ELF (no libc)
--compile-uefi FILE -o OUT	UEFI PE application
--emit-kotlin FILE -o OUT	Emit Kotlin source code
--compile-android FILE -o DIR	Generate Android Gradle project
--target ARCH	Target architecture: x86-64, aarch64, rv64

Android Options:

--package PKG	Android package name (default: com.sage.app)
--app-name NAME	Display name for Android app
--min-sdk N	Minimum Android SDK version (default: 24)

Runtime:

--runtime MODE	Execution backend: ast, bytecode, jit, aot, auto
--math-work=MODES	Comma-separated list of math visualization formats: grade, exec, bitwis
e	
--jit FILE	JIT compile with profiling
--aot FILE	AOT compile to native binary

Garbage Collection:

--gc:tracing	Tracing GC (default, concurrent mark-sweep)
--gc:arc	Automatic Reference Counting
--gc:orc	Optimized RC with cycle detection

Optimization:

-00	No optimization
-01	Constant folding
-02	Constant folding + DCE + type specialization (Kotlin)
-03	Aggressive (+ inlining)
-g	Include debug info

Safety:

--safety	Check @safe annotated functions
--strict-safety	Enforce safety globally

Tooling:

--fmt FILE	Format source code
--lint FILE	Lint source code
--lsp	Start Language Server Protocol server
fmt FILE	Format source code
lint FILE	Lint source code
check FILE	Run type checker
safety FILE	Run safety analyzer

Self-Hosted Compiler:

sage.sage FILE	Run file in self-hosted interpreter
sage.sage --emit-c FILE	Emit C via self-hosted compiler
sage.sage --emit-llvm FILE	Emit LLVM IR via self-hosted compiler
sage.sage --emit-asm FILE	Emit assembly via self-hosted compiler
sage.sage fmt FILE	Format via self-hosted formatter
sage.sage lint FILE	Lint via self-hosted linter
sage.sage check FILE	Type check via self-hosted checker

REPL Commands:

:help	Show REPL help
:env	Show environment
:ast <code>	Show parsed AST
:emit-c <code>	Show C backend output
:emit-llvm <code>	Show LLVM IR output
:emit-kotlin <code>	Show Kotlin backend output

<code>:time <expr></code>	Time expression evaluation
<code>:bench N <expr></code>	Benchmark expression N times
<code>:stats</code>	Show interpreter and GC statistics
<code>:runtime MODE</code>	Switch runtime (ast, bytecode, jit, aot)

\newpage

Appendix C: Language Gotchas

This section documents known behaviors and design decisions. Items marked **FIXED** were resolved in v1.4 or later.

Still Relevant

- **<0 is truthy>** -- Only `false` and `nil` are falsy. This is an intentional design decision. Use explicit comparisons: `if x == 0:` not `if not x:`.

- **<chr(0) truncates strings>** -- Null bytes terminate C strings in the interpreter. The runtime uses C-style null-terminated strings, so inserting `chr(0)` into a string will truncate it at that point.

- **<No wildcard imports>** -- `from module import *` is not supported. Use `import module` then `module.func()`, or import specific names: `from module import func`.

- **<No multiline dict/array literals>** -- Build complex structures incrementally with assignments:

```
let config = {}
config["host"] = "localhost"
config["port"] = 8080
```

- **<match, init, end, enum, struct, trait>** -- **FIXED.** As of v4.1.5, `match`, `init`, `end`, `enum`, `struct`, and `trait` can be used as variable names in expression and assignment contexts, as long as they are not immediately followed by a block-starting colon or used in a declaration context.
- **<super auto-injects self>** -- **FIXED.** As of v4.1.5, you can write either `super.init(args)` or `super.init(self, args)`. The interpreter and backends now handle both forms correctly.

Fixed in v1.4+

- **<No escape sequences>** -- **FIXED.** `\n`, `\t`, `\r`, `\\`

size="8.4">\",</p>
</div>
<div data-bbox="95 80 908 112" data-label="Text">
<p>\xHH and other escape sequences now work in string literals.</p>
</div>
<div data-bbox="118 120 886 169" data-label="List-Group">

• ~~No hex literals~~ -- FIXED. 0xFF and 0o755 are now parsed

</div>
<div data-bbox="95 174 338 189" data-label="Text">
<p>correctly as number literals.</p>
</div>
<div data-bbox="118 197 888 229" data-label="List-Group">

• ~~elif chains malfunction~~ -- FIXED. Unlimited elif branches

</div>
<div data-bbox="95 233 387 249" data-label="Text">
<p>are now supported without issues.</p>
</div>
<div data-bbox="118 256 862 272" data-label="List-Group">

• ~~Instance == always false~~ -- FIXED. Structural equality now

</div>
<div data-bbox="95 275 466 291" data-label="Text">
<p>works for class instances, arrays, and dicts.</p>
</div>
<div data-bbox="118 298 886 332" data-label="List-Group">

• ~~push/append mismatch~~ -- FIXED. Both push() and append()

</div>
<div data-bbox="95 335 796 352" data-label="Text">
<p>work identically in the C interpreter. They are aliases for the same native function.</p>
</div>
<div data-bbox="118 358 888 392" data-label="List-Group">

• ~~% casts to int~~ -- FIXED. The modulo operator now uses fmod()

</div>
<div data-bbox="95 395 908 428" data-label="Text">
<p>and preserves float semantics. 3.7 % 1 returns 0.7, not 0.</p>
</div>
<div data-bbox="118 435 886 467" data-label="List-Group">

• ~~Class methods can't see module-level vars~~ -- FIXED.
Methods

</div>
<div data-bbox="95 472 906 505" data-label="Text">
<p>now have access to their defining environment via defining_env.</p>
</div>
<div data-bbox="118 512 886 544" data-label="List-Group">

• ~~Circular Imports~~ -- FIXED. Circular dependencies now
resolve

</div>
<div data-bbox="95 549 583 564" data-label="Text">
<p>correctly by providing access to partially initialized state.</p>
</div>
<div data-bbox="118 571 888 604" data-label="List-Group">

• ~~LLVM Loop Mutation~~ -- FIXED. Modifying a for loop variable

</div>
<div data-bbox="95 607 649 624" data-label="Text">
<p>inside the loop now correctly affects the iteration count in LLVM.</p>
</div>
<div data-bbox="118 631 856 667" data-label="List-Group">

• ~~No string interning~~ -- FIXED. Sage now implements global
string interning to reduce memory usage and speed up comparisons.

</div>
<div data-bbox="95 674 840 710" data-label="Text">
<p>• ~~AST Interpreter Performance~~ -- FIXED. The tree-walking
interpreter has been optimized with "Quickened" nodes for numeric paths.</p>
</div>
<div data-bbox="48 718 135 734" data-label="Text">
<p>\newpage</p>
</div>
<div data-bbox="413 953 487 969" data-label="Page-Footer">
Page 103
</div>

Appendix D: Reserved Keywords

Sage reserves the following 54 keywords. They cannot be used as variable or function names.

Control Flow

K e y w o r d	P u r p o s e
, i f	C o n d i t i o n a l b r a
n c h	E l s e - i f b r a n c h
, e l l i f	D e f a u l t b r a n c h
, e l l s e	W h i l e l o o p
, w h i l e	F o r - i n l o o p
, f o r	F o r - i n i t e r a t i o
n , i n	E x i t l o o p
, b r e a k	S k i p t o n e x t i t
, c o n t i n u e	P a t t e r n m a t c h i n
e r a t i o n	M a t c h c a s e
, m a t c h	M a t c h d e f a u l t
, c a s e	B l o c k t e r m i n a t o
, d e f a u l t	
, e n d	
r	

Functions and Classes

K e y w o r d	P u r p o s e
, p r o c	F u n c t i o n d e f i n i
t i o n	R e t u r n f r o m f u n
, r e t u r n	C l a s s d e f i n i t i o
c t i o n	I n s t a n c e r e f e r e
, c l l a s s	C o n s t r u c t o r m e t
n , s e l f	P a r e n t c l l a s s r e
, s e l f	S t r u c t d e f i n i t i
n c e i n i t	E n u m d e f i n i t i o n
, i n i t	T r a i t d e f i n i t i o
h o d	
, s u p e r	
f e r e n c e	
, s t r u c t	
o n	
, e n u m	
, t r a i t	
n	

Variables

K e y w o r d	P u r p o s e
, l e t	I m m u t a b l e b i n d i
n g	M u t a b l e v a r i a b l
, v a r	
e	

Values and Logic

K e y w o r d	P u r p o s e
---------------------------	---------------------------

[illegible][illegible]

Page 105